

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	R . ASHWITHA REDDY
ROLL NO	AV.SC.U4CSE25238
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	

5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12) Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10) Exit	

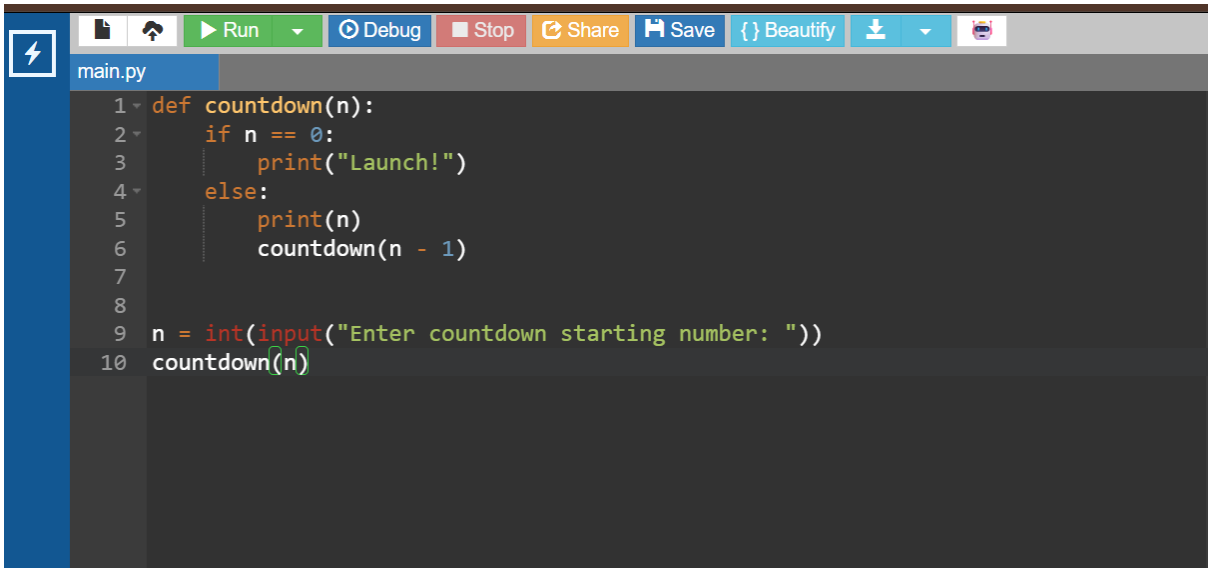
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes	

	5. Count Internal Nodes 6. Height of the Tree	
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	

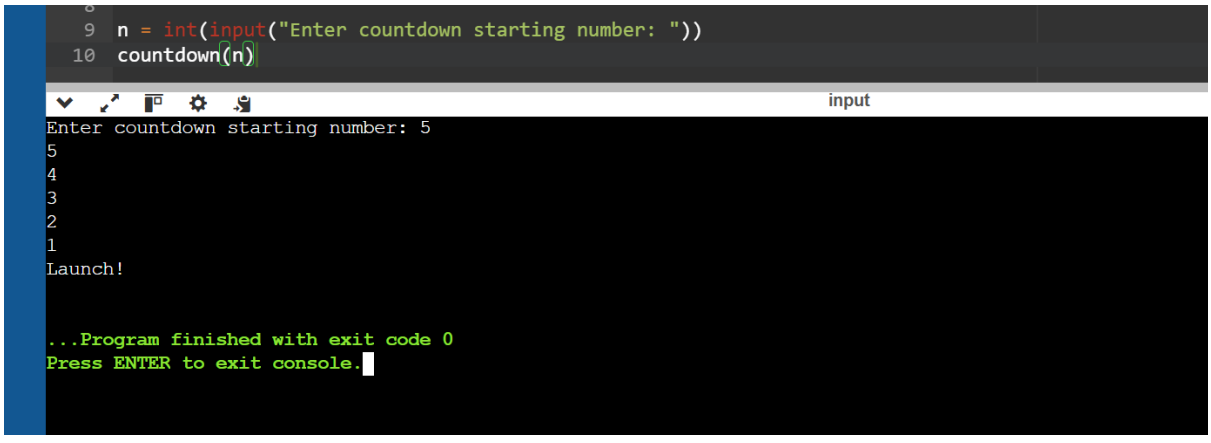
WEEK 1:

1(A). A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by “Launch!”.

PROGRAM:

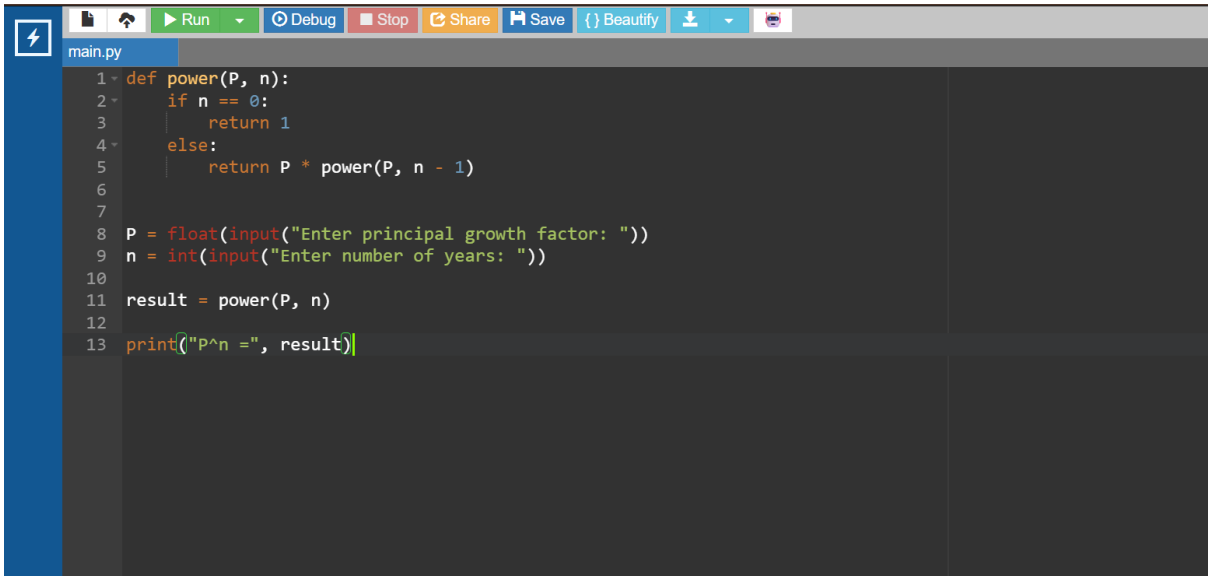


OUTPUT:

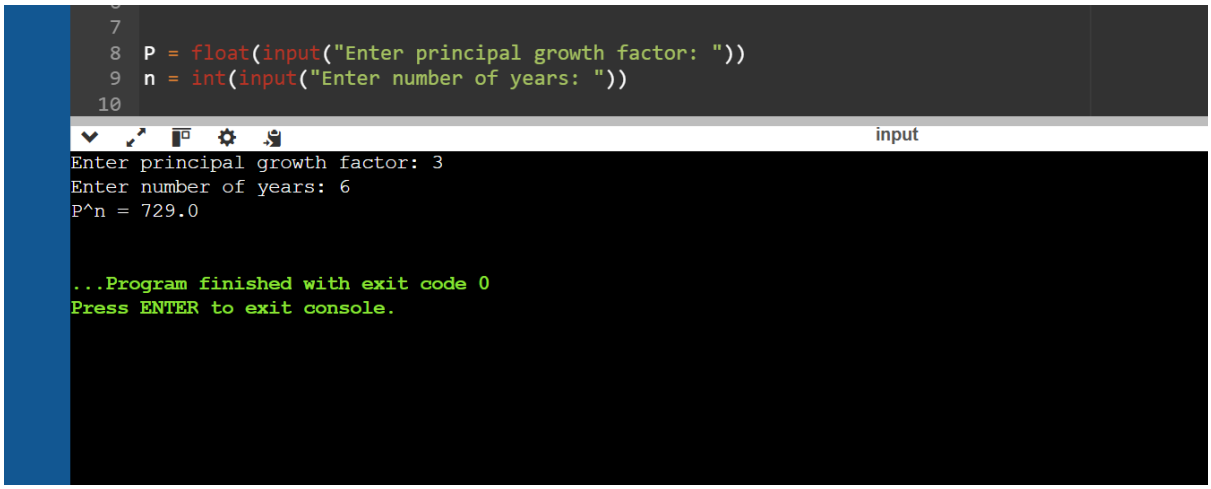


1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

PROGRAM:

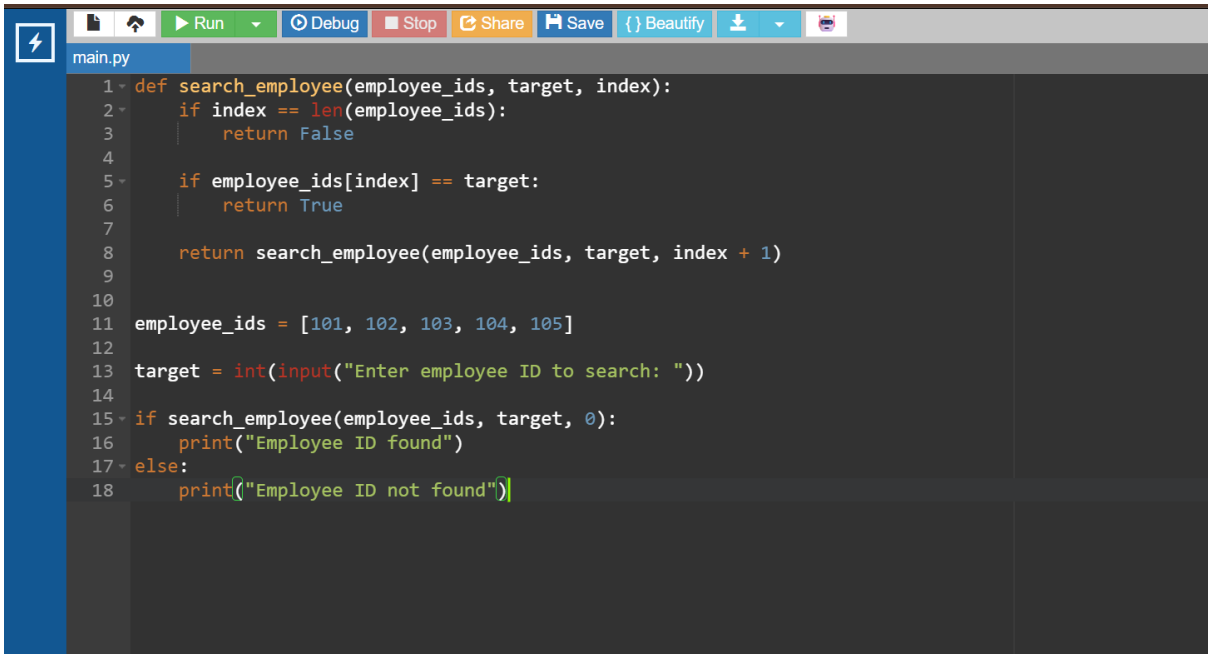


OUTPUT:

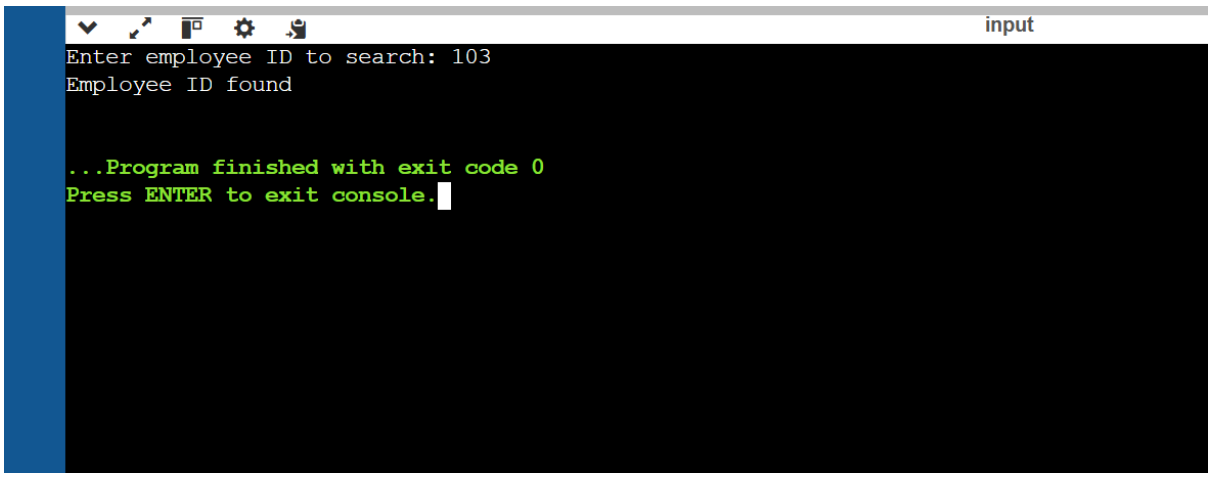


1(C). A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:



OUTPUT:

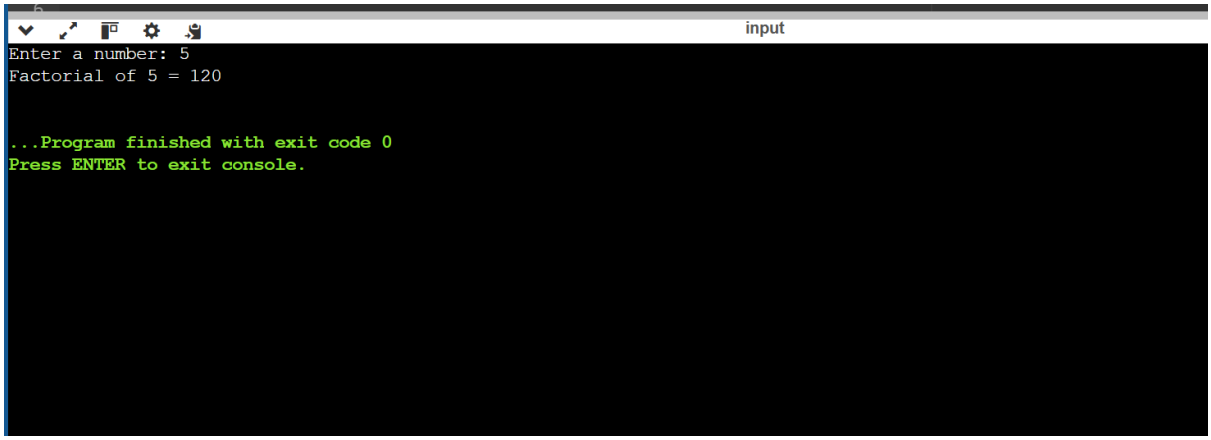


1(D). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:



OUTPUT:

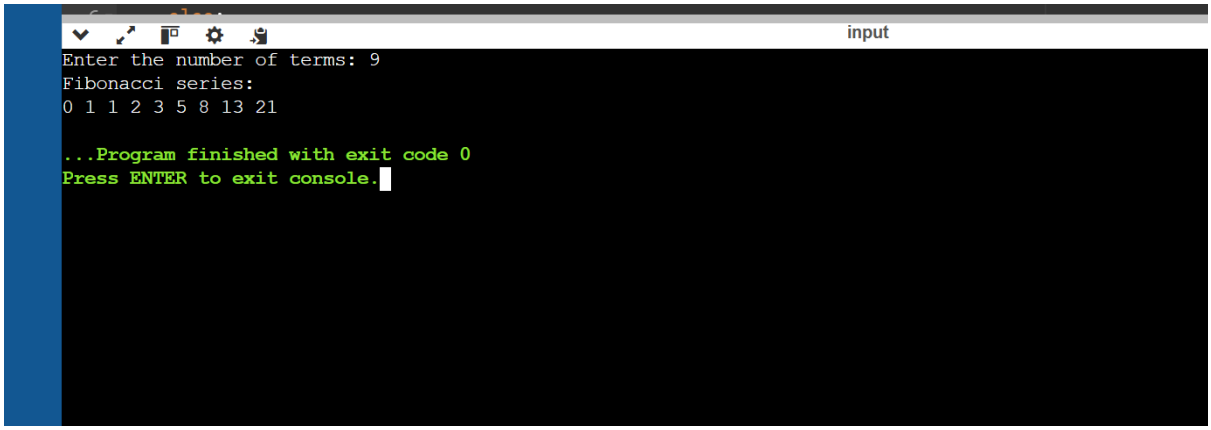


1(E). Write a recursive python program to print the first n terms of the Fibonacci series

PROGRAM:



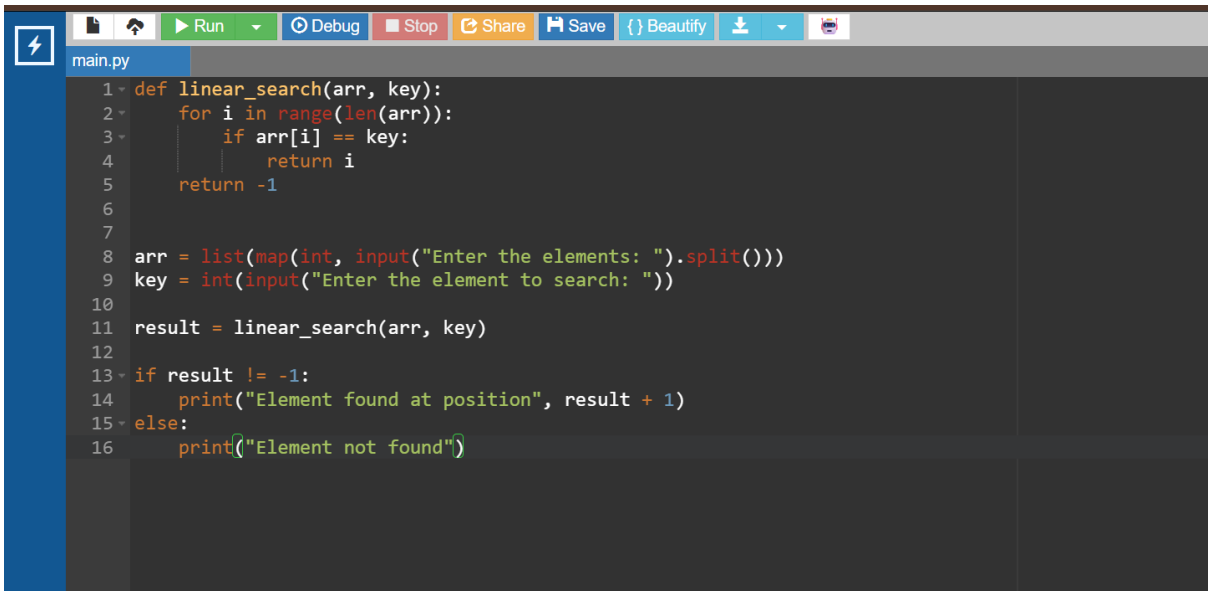
OUTPUT:



WEEK 2:

2(A). Write a python program to implement linear search.

PROGRAM:

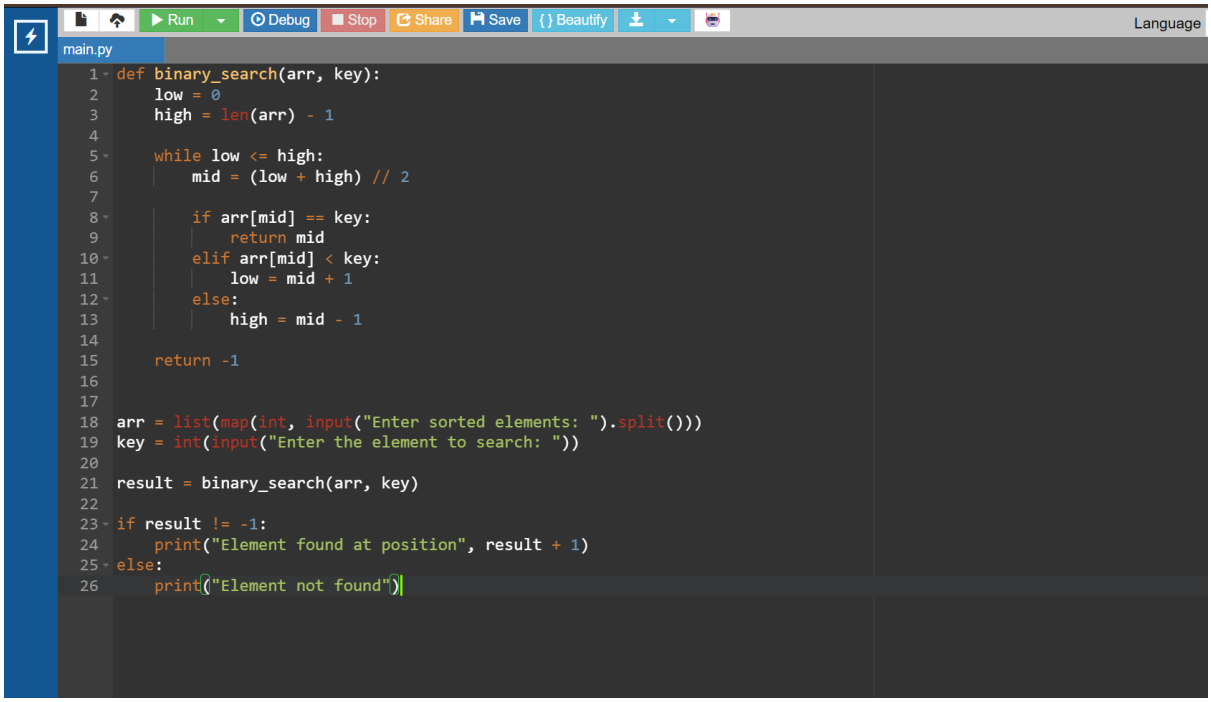


OUTPUT:

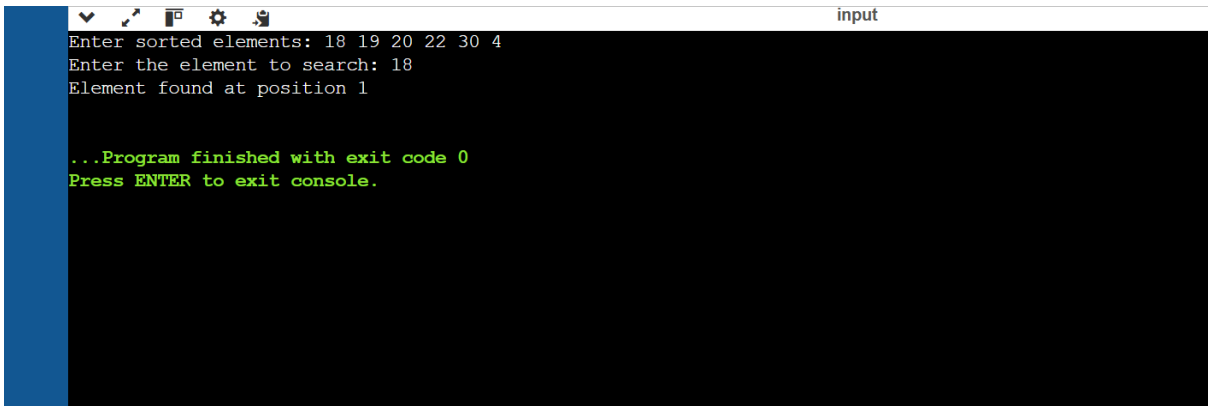


2(B). Write a python program to implement binary search with and without sorted array as an input

PROGRAM:



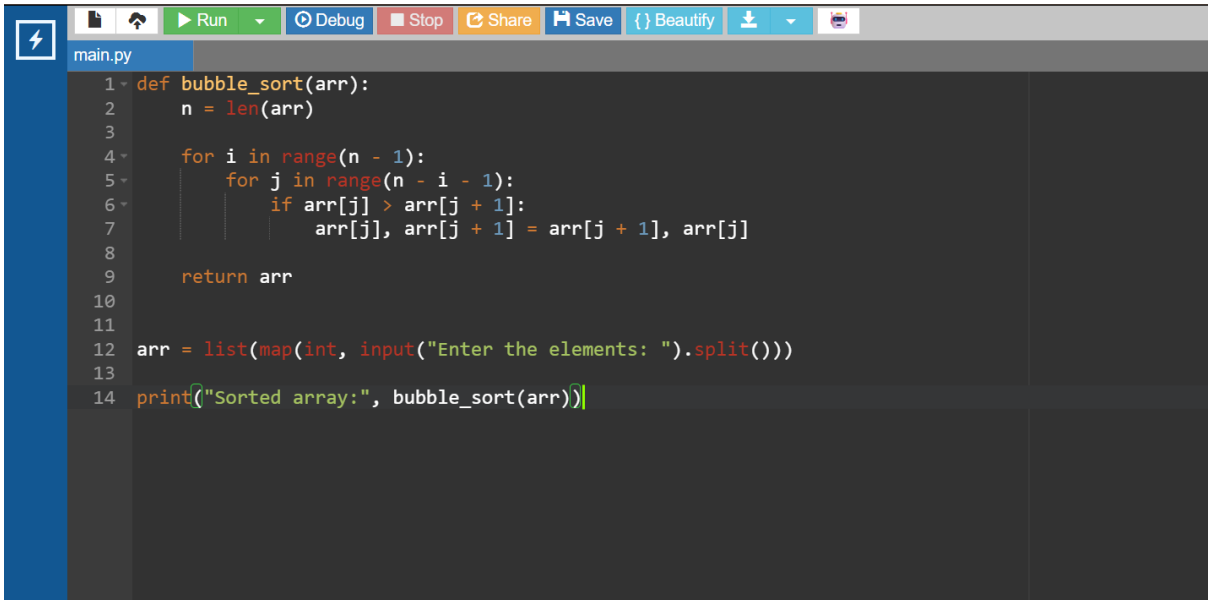
OUTPUT:



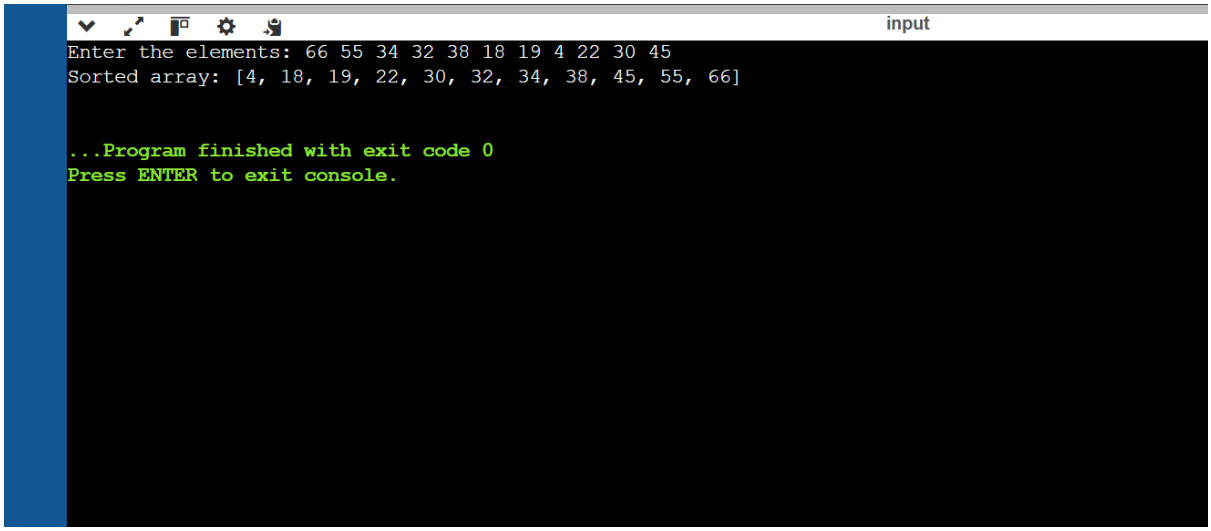
WEEK-3:

3(A). Write a python program to implement bubble sort.

PROGRAM:



OUTPUT:

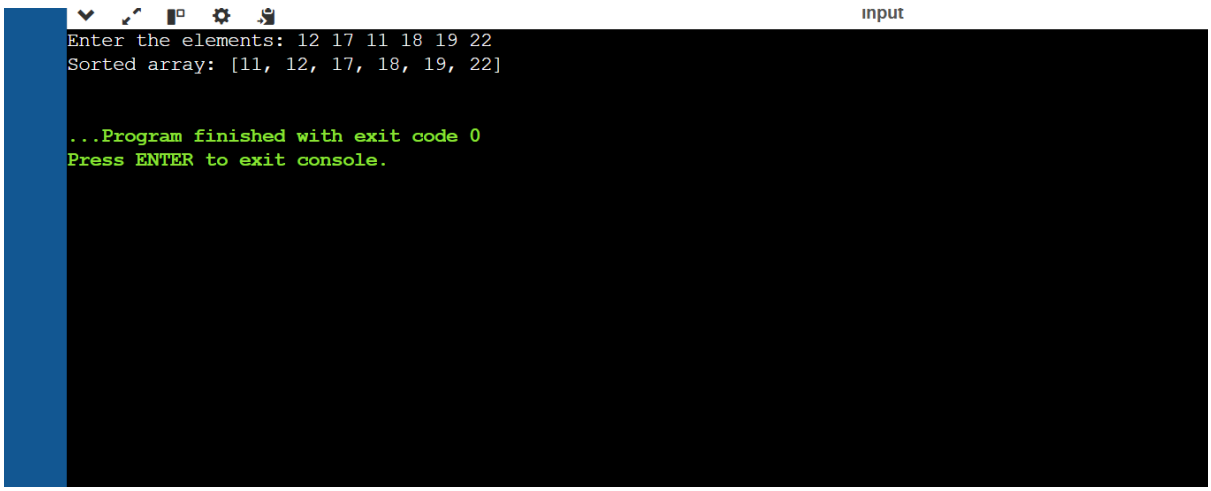


3(B). Write a python program to implement insertion sort.

PROGRAM:

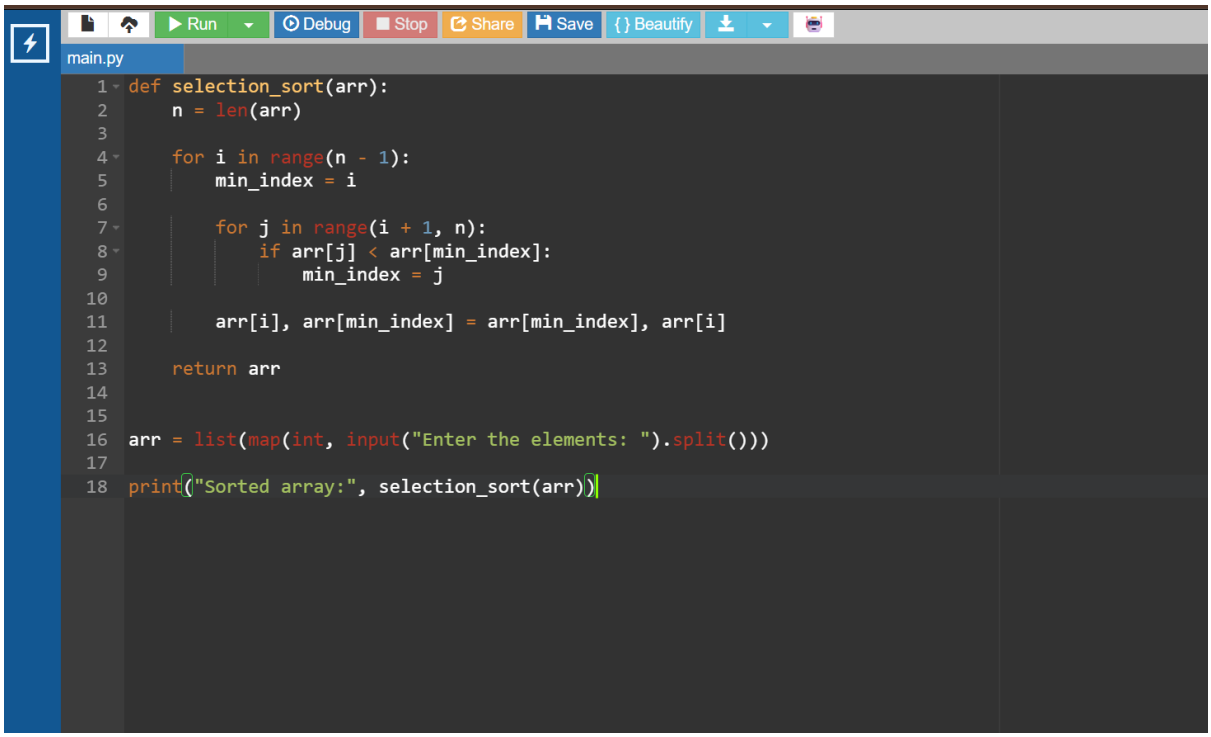


OUTPUT:

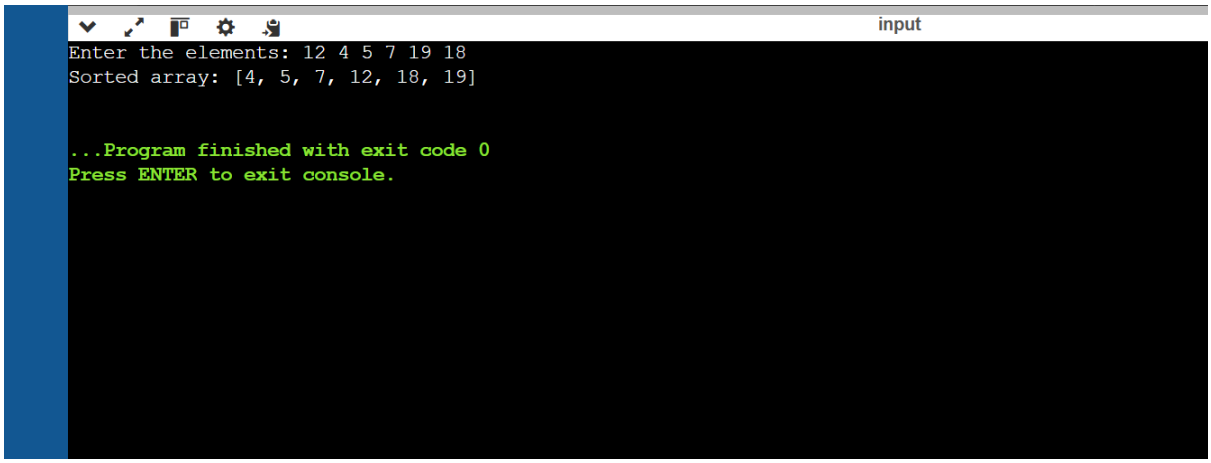


3(C). Write a python program to implement selection sort.

PROGRAM:



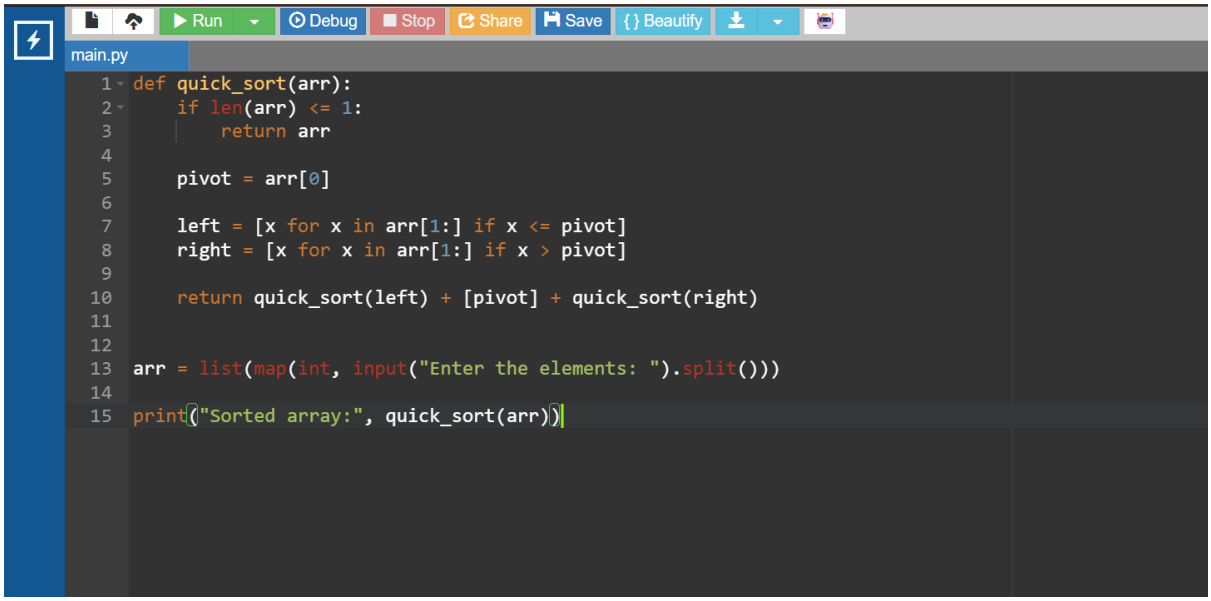
OUTPUT:



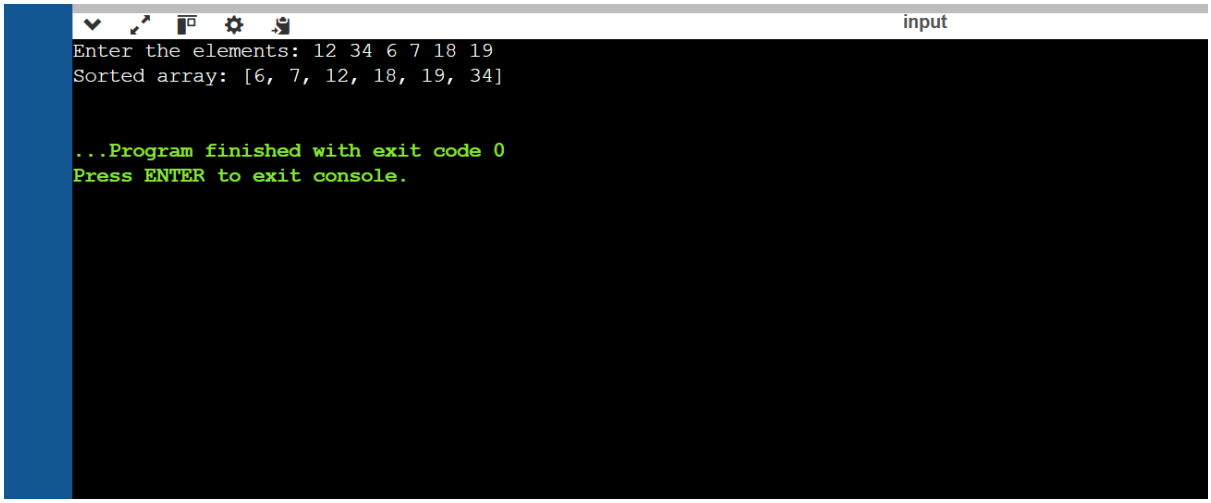
WEEK-4:

4(A). Write a python program to implement quick sort.

PROGRAM:



OUTPUT:

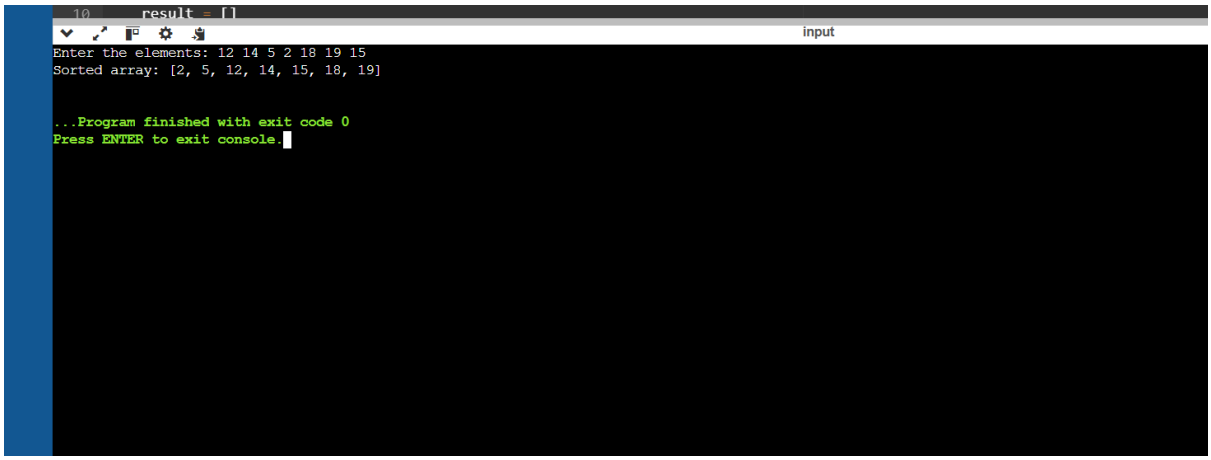


4(B). Write a python program to implement merge sort.

PROGRAM:



OUTPUT:

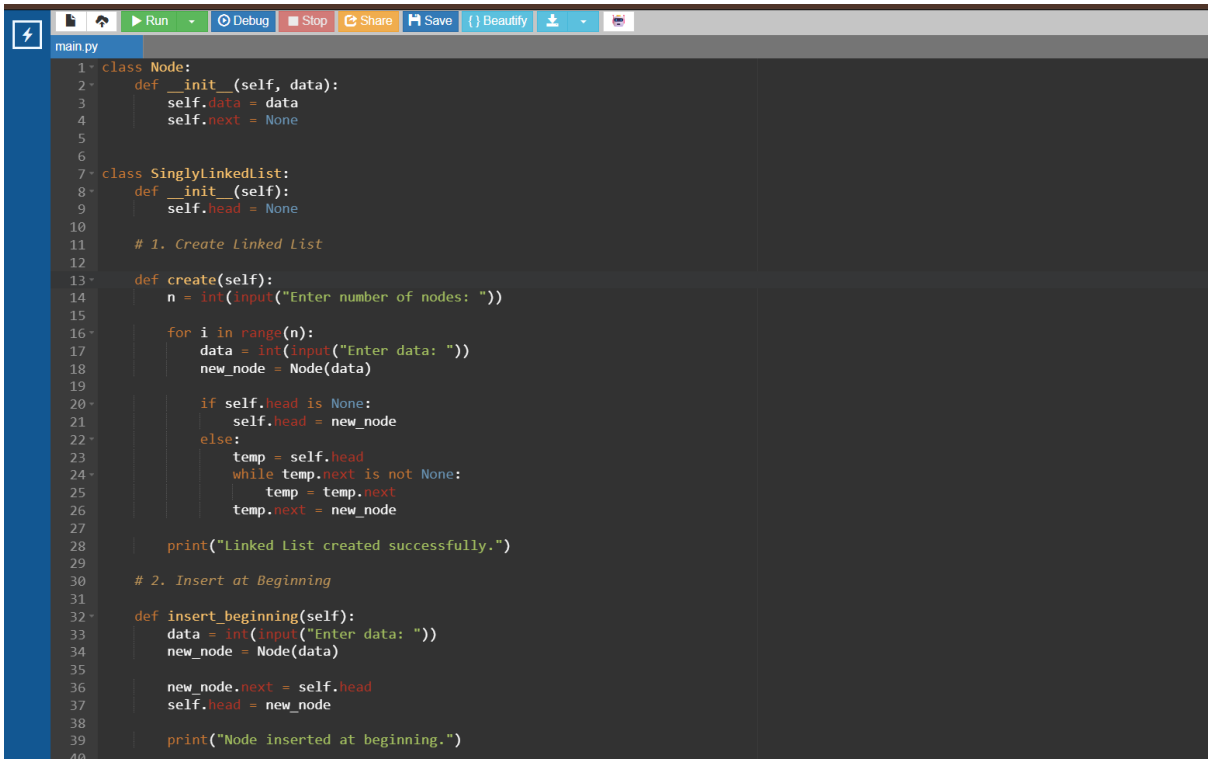


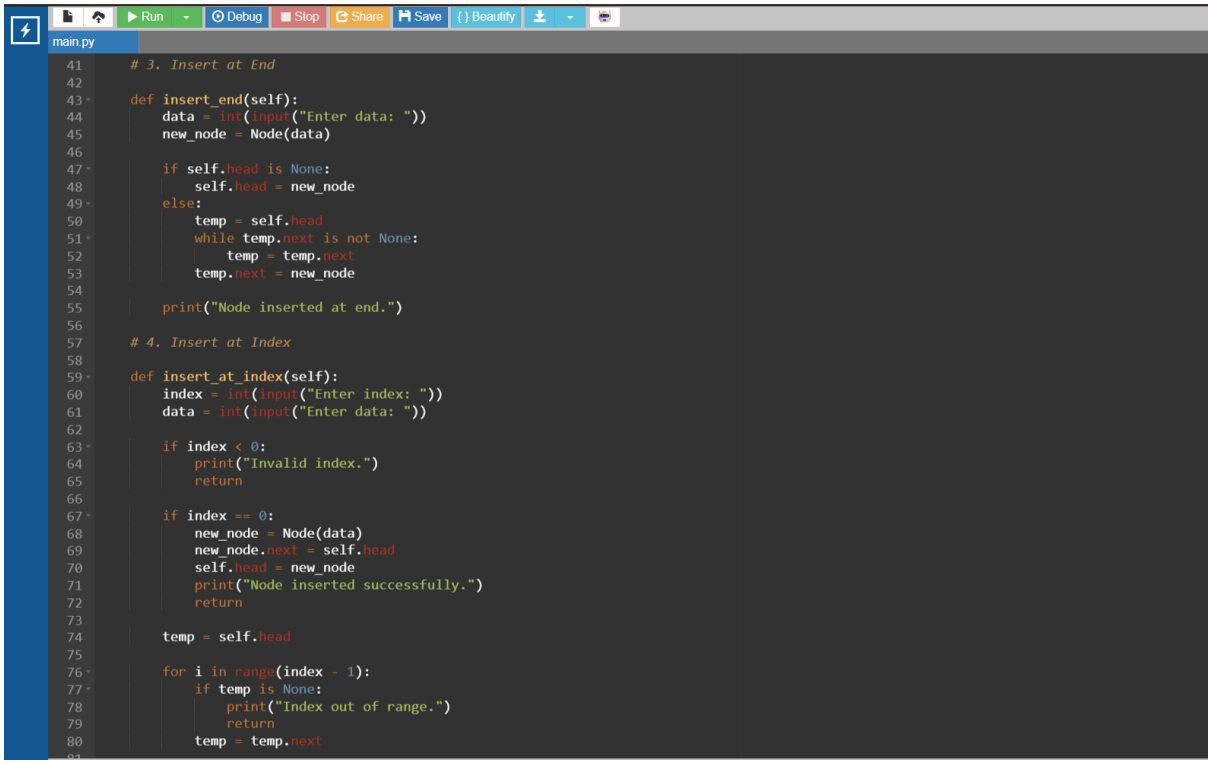
WEEK-5:

5(A). Write a python program to implement a Singly Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Sort the data of SLL
10) Search data in SLL
11) Display or Traverse
12) Exit

PROGRAM:






```
    # 3. Insert at End

    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next is not None:
                temp = temp.next
            temp.next = new_node

        print("Node inserted at end.")

    # 4. Insert at Index

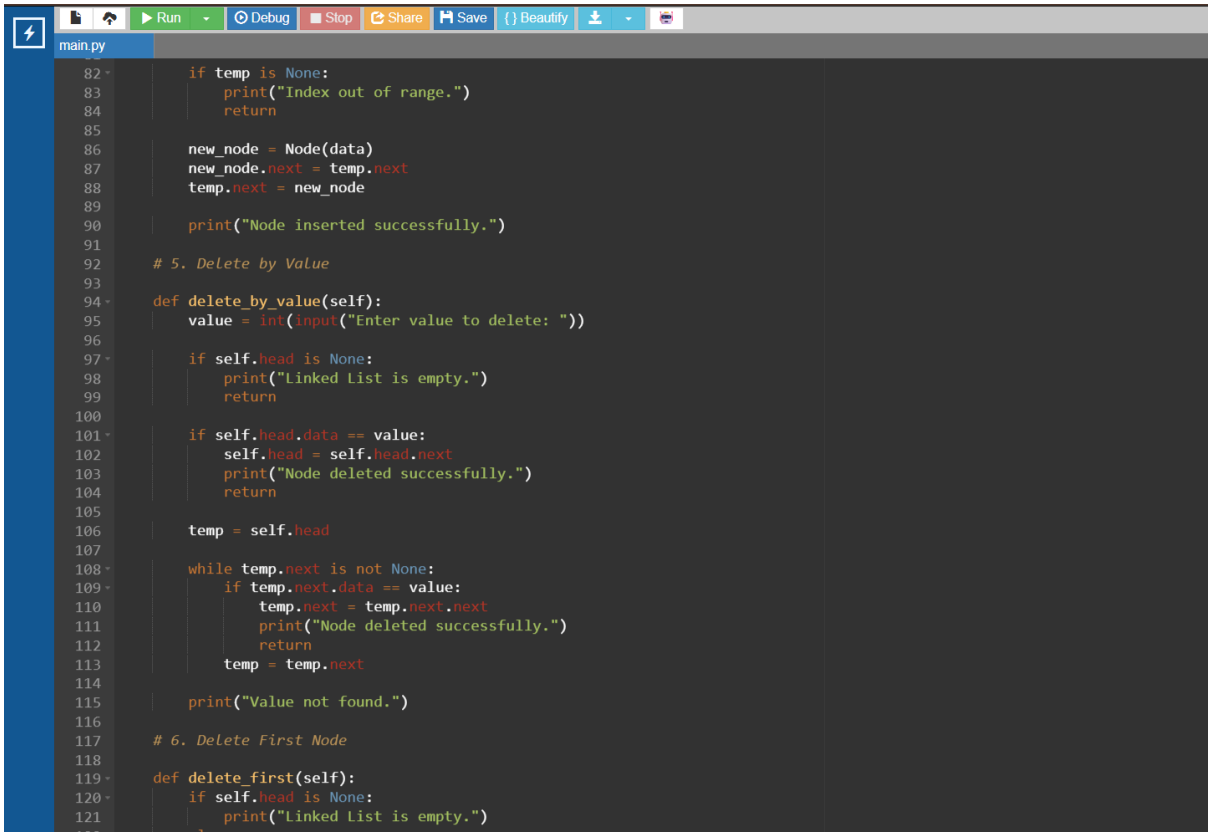
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        if index < 0:
            print("Invalid index.")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            print("Node inserted successfully.")
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return
            temp = temp.next
```




```
        if temp is None:
            print("Index out of range.")
            return

        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted successfully.")

    # 5. Delete by Value

    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("Linked List is empty.")
            return

        if self.head.data == value:
            self.head = self.head.next
            print("Node deleted successfully.")
            return

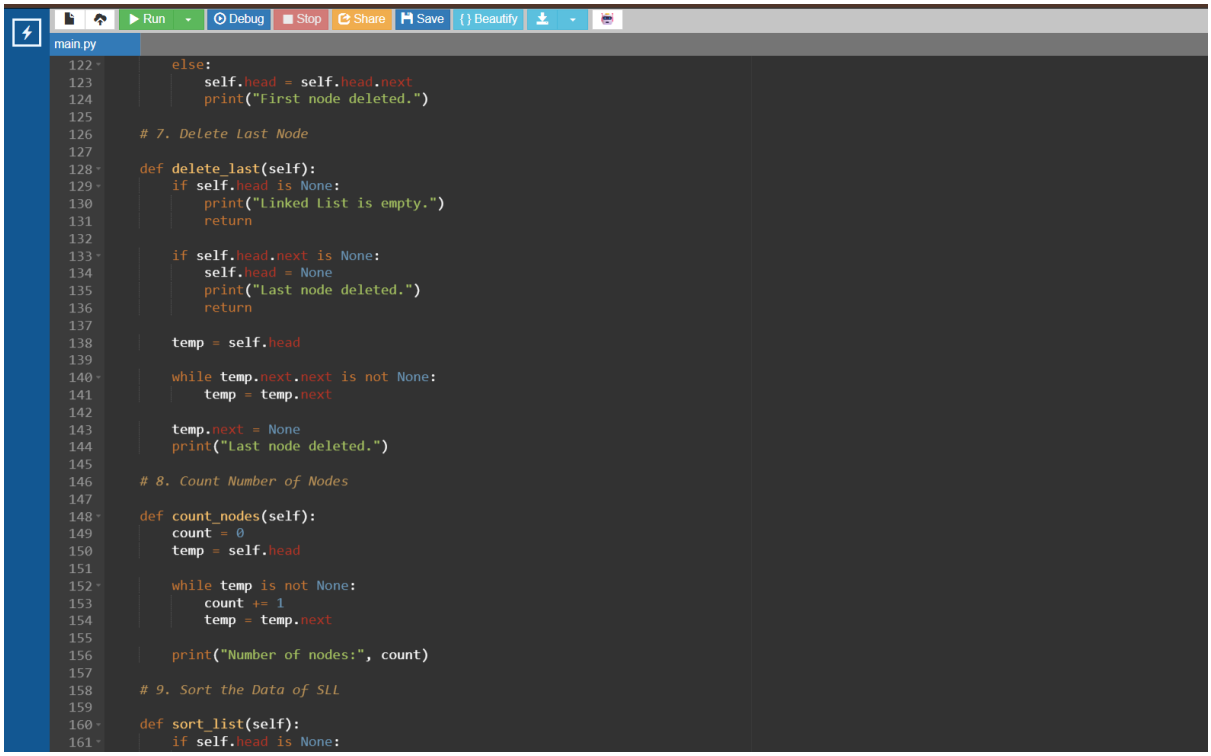
        temp = self.head

        while temp.next is not None:
            if temp.next.data == value:
                temp.next = temp.next.next
                print("Node deleted successfully.")
                return
            temp = temp.next

        print("Value not found.")

    # 6. Delete First Node

    def delete_first(self):
        if self.head is None:
            print("Linked List is empty.")
```




```
        else:
            self.head = self.head.next
            print("First node deleted.")

    # 7. Delete Last Node

    def delete_last(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        if self.head.next is None:
            self.head = None
            print("Last node deleted.")
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None
        print("Last node deleted.")

    # 8. Count Number of Nodes

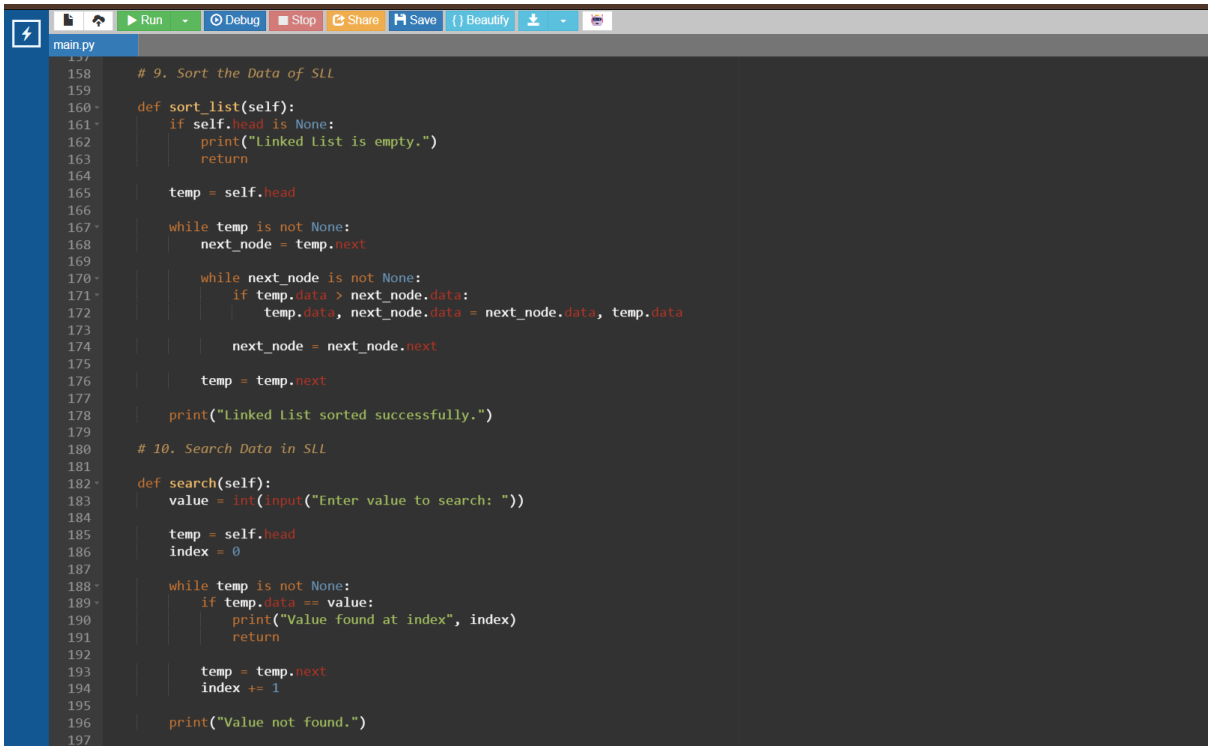
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # 9. Sort the Data of SLL

    def sort_list(self):
        if self.head is None:
```




```
    # 9. Sort the Data of SLL

    def sort_list(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head

        while temp is not None:
            next_node = temp.next

            while next_node is not None:
                if temp.data > next_node.data:
                    temp.data, next_node.data = next_node.data, temp.data

                next_node = next_node.next

            temp = temp.next

        print("Linked List sorted successfully.")

    # 10. Search Data in SLL

    def search(self):
        value = int(input("Enter value to search: "))

        temp = self.head
        index = 0

        while temp is not None:
            if temp.data == value:
                print("Value found at index", index)
                return

            temp = temp.next
            index += 1

        print("Value not found.")
```

Run Debug Stop Share Save { } Beautify

main.py

```
    # 11. Display / Traverse

    def display(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


    # Main Program

sll = SinglyLinkedList()

while True:
    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Sort the Data")
    print("10. Search Data")
    print("11. Display / Traverse")
    print("12. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()
```

Run Debug Stop Share Save { } Beautify

main.py

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()

    elif choice == 2:
        sll.insert_beginning()

    elif choice == 3:
        sll.insert_end()

    elif choice == 4:
        sll.insert_at_index()

    elif choice == 5:
        sll.delete_by_value()

    elif choice == 6:
        sll.delete_first()

    elif choice == 7:
        sll.delete_last()

    elif choice == 8:
        sll.count_nodes()

    elif choice == 9:
        sll.sort_list()

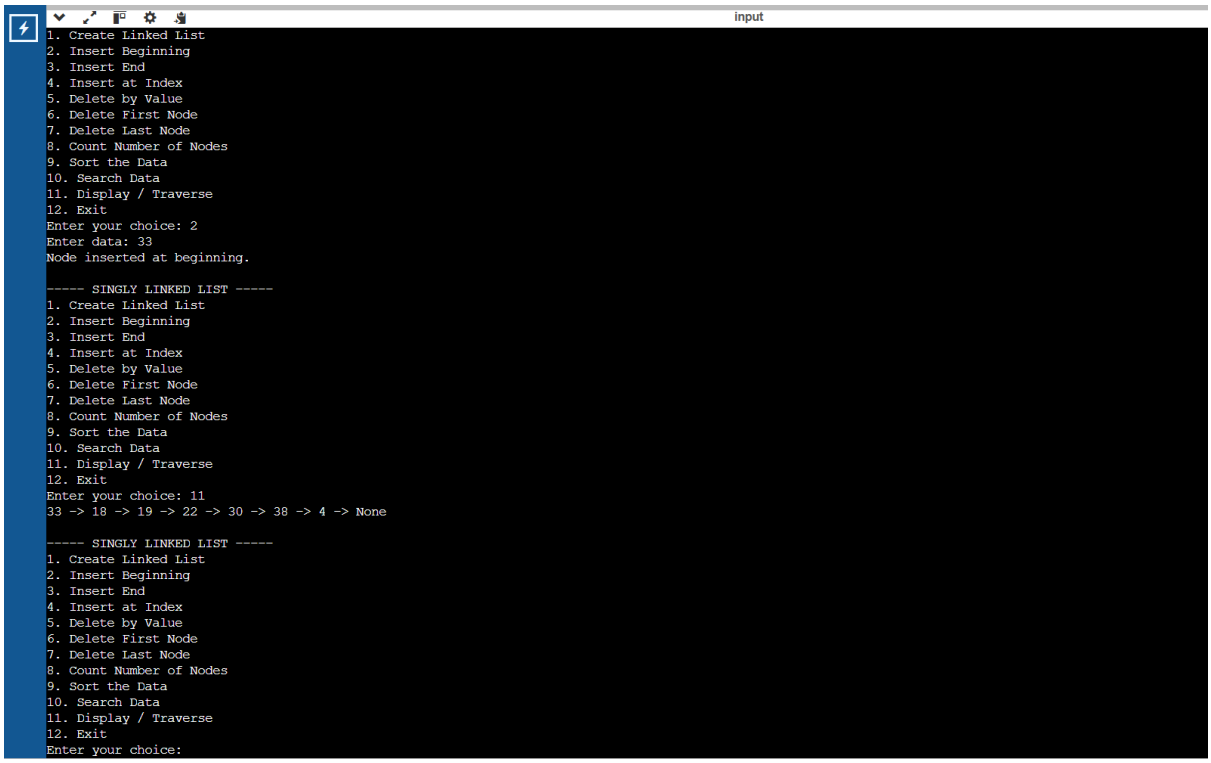
    elif choice == 10:
        sll.search()

    elif choice == 11:
        sll.display()

    elif choice == 12:
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

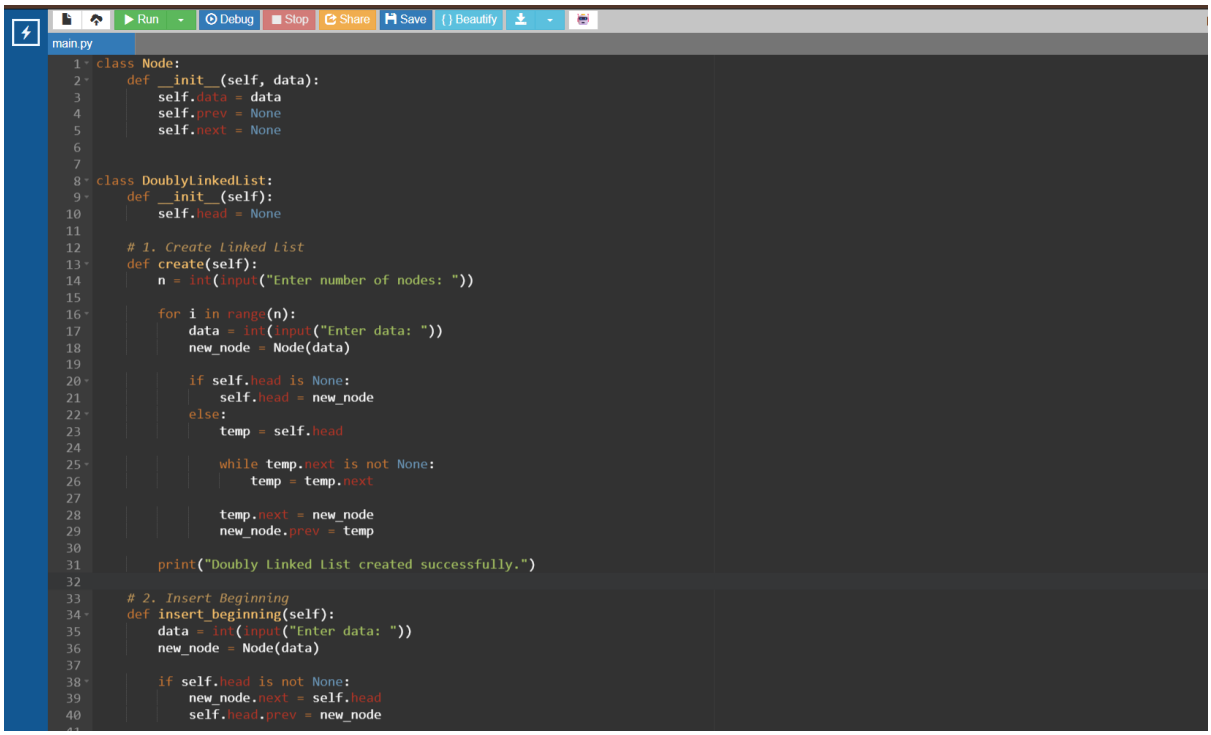


WEEK-6:

Write a python program to implement a Double Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Display or Traverse
10) Exit

PROGRAM:




```python
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

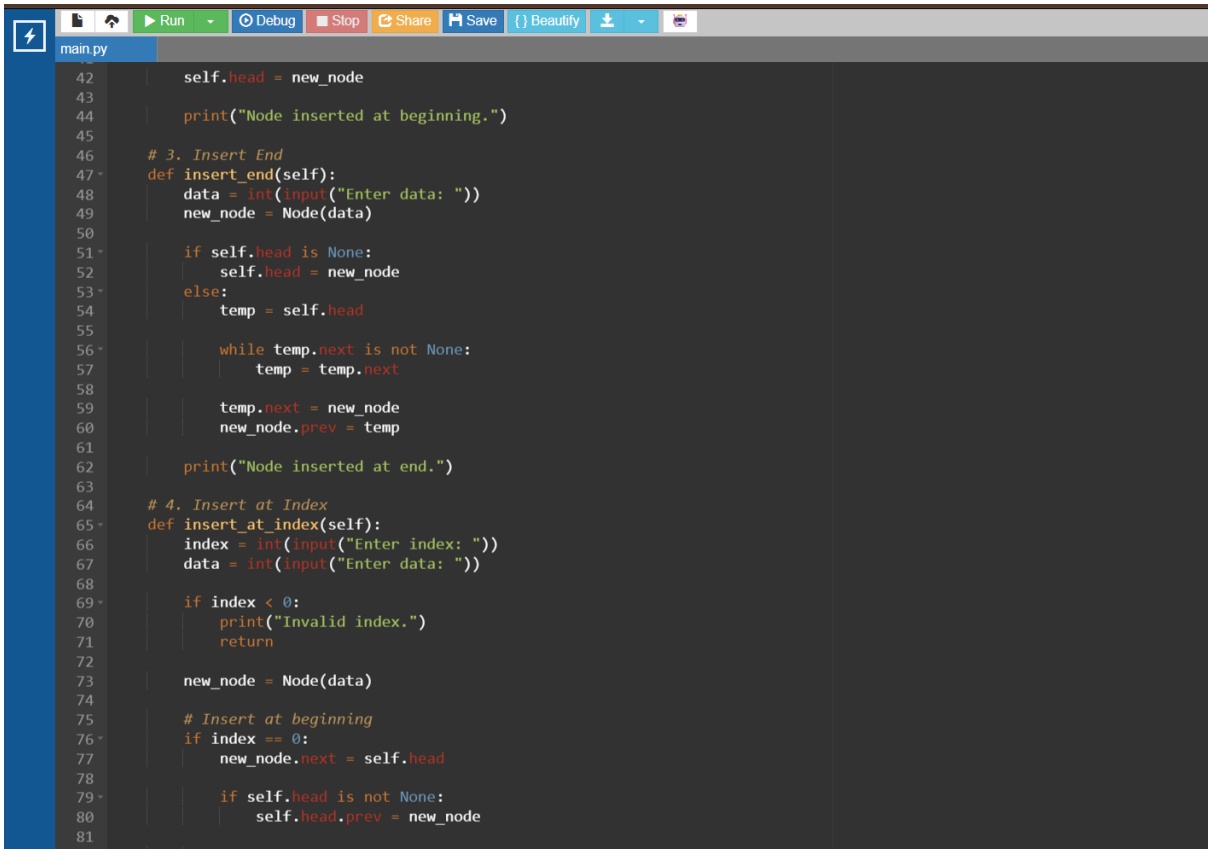
                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp

        print("Doubly Linked List created successfully.")

    # 2. Insert Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node
```




```python
        self.head = new_node

        print("Node inserted at beginning.")

    # 3. Insert End
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node inserted at end.")

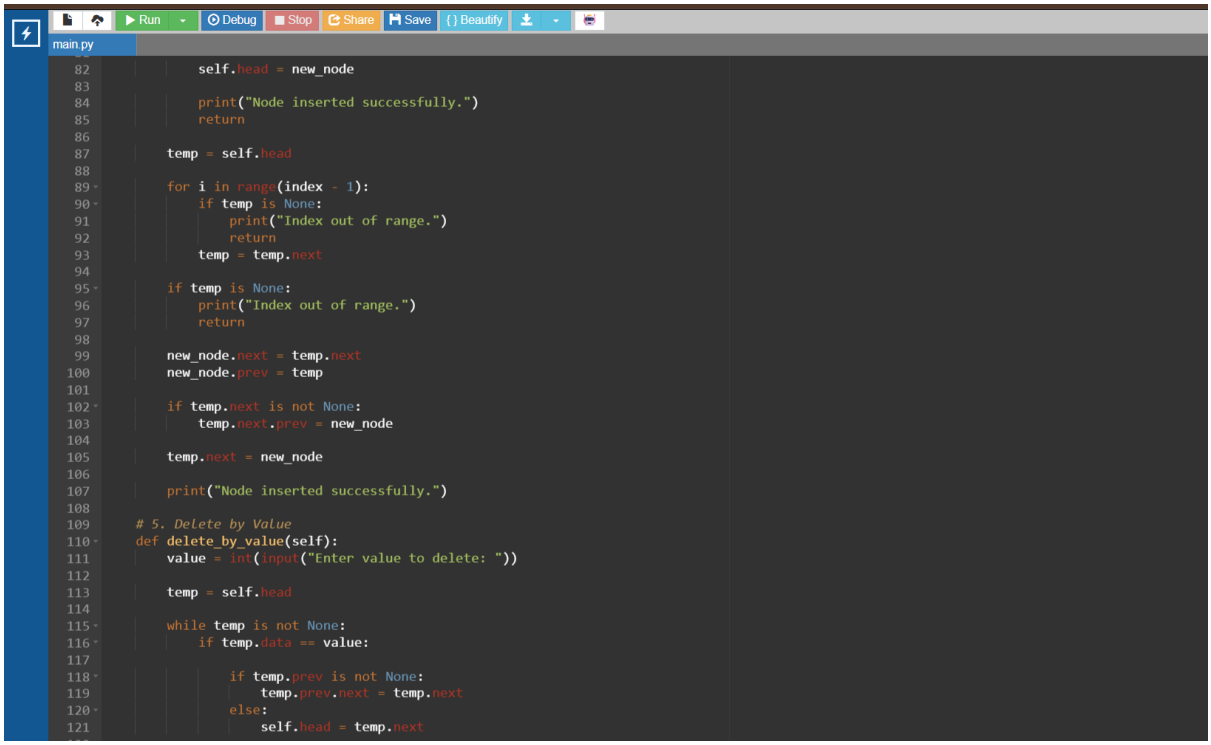
    # 4. Insert at Index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        if index < 0:
            print("Invalid index.")
            return

        new_node = Node(data)

        # Insert at beginning
        if index == 0:
            new_node.next = self.head

            if self.head is not None:
                self.head.prev = new_node
```




```
            self.head = new_node

            print("Node inserted successfully.")
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return
            temp = temp.next

        if temp is None:
            print("Index out of range.")
            return

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

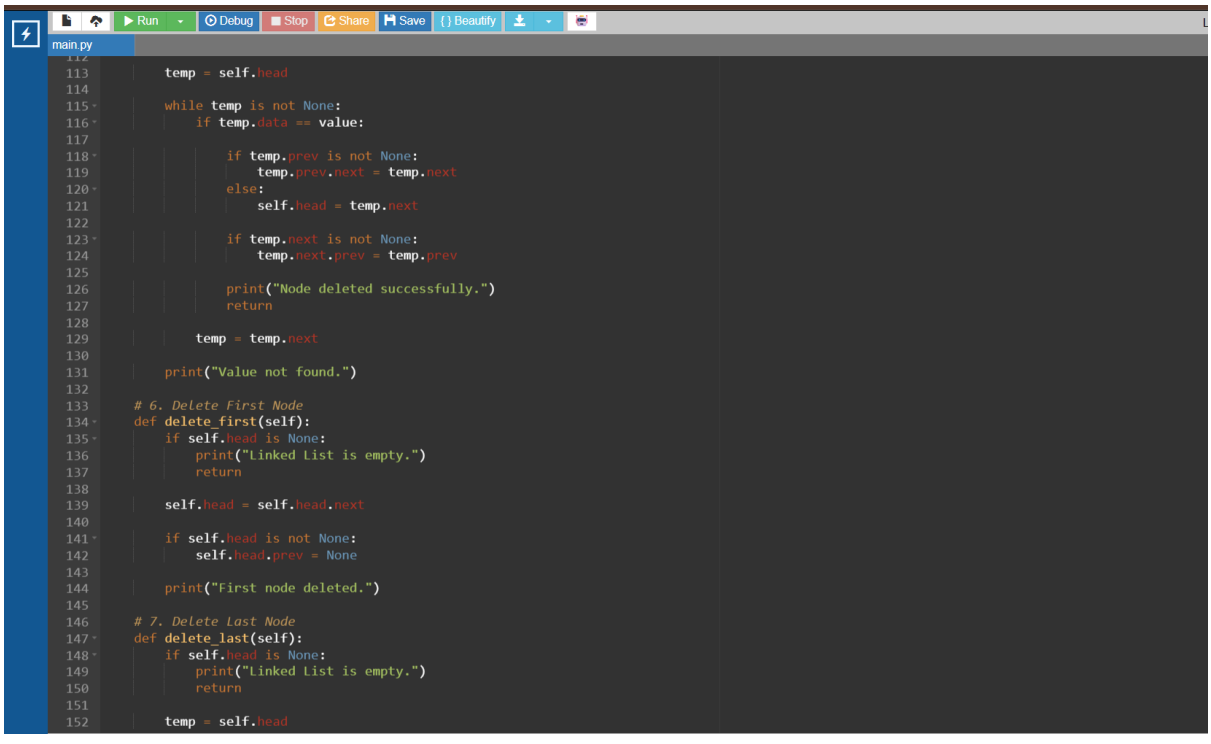
        print("Node inserted successfully.")

    # 5. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        temp = self.head

        while temp is not None:
            if temp.data == value:

                if temp.prev is not None:
                    temp.prev.next = temp.next
                else:
                    self.head = temp.next
```




```
        temp = self.head

        while temp is not None:
            if temp.data == value:

                if temp.prev is not None:
                    temp.prev.next = temp.next
                else:
                    self.head = temp.next

                if temp.next is not None:
                    temp.next.prev = temp.prev

                print("Node deleted successfully.")
                return

            temp = temp.next

        print("Value not found.")

    # 6. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("First node deleted.")

    # 7. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head
```

Run | Debug | Stop | Share | Save | { } Beautify

main.py

```
        while temp.next is not None:
            temp = temp.next

        if temp.prev is None:
            self.head = None
        else:
            temp.prev.next = None

        print("Last node deleted.")

    # 8. Count Number of Nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head

        print("Doubly Linked List:")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")
```

Run | Debug | Stop | Share | Save | { } Beautify

main.py

```
# Main Program
dll = DoublyLinkedList()

while True:
    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()

    elif choice == 4:
        dll.insert_at_index()

    elif choice == 5:
        dll.delete_by_value()

    elif choice == 6:
        dll.delete_first()

    elif choice == 7:
        dll.delete_last()
```

```

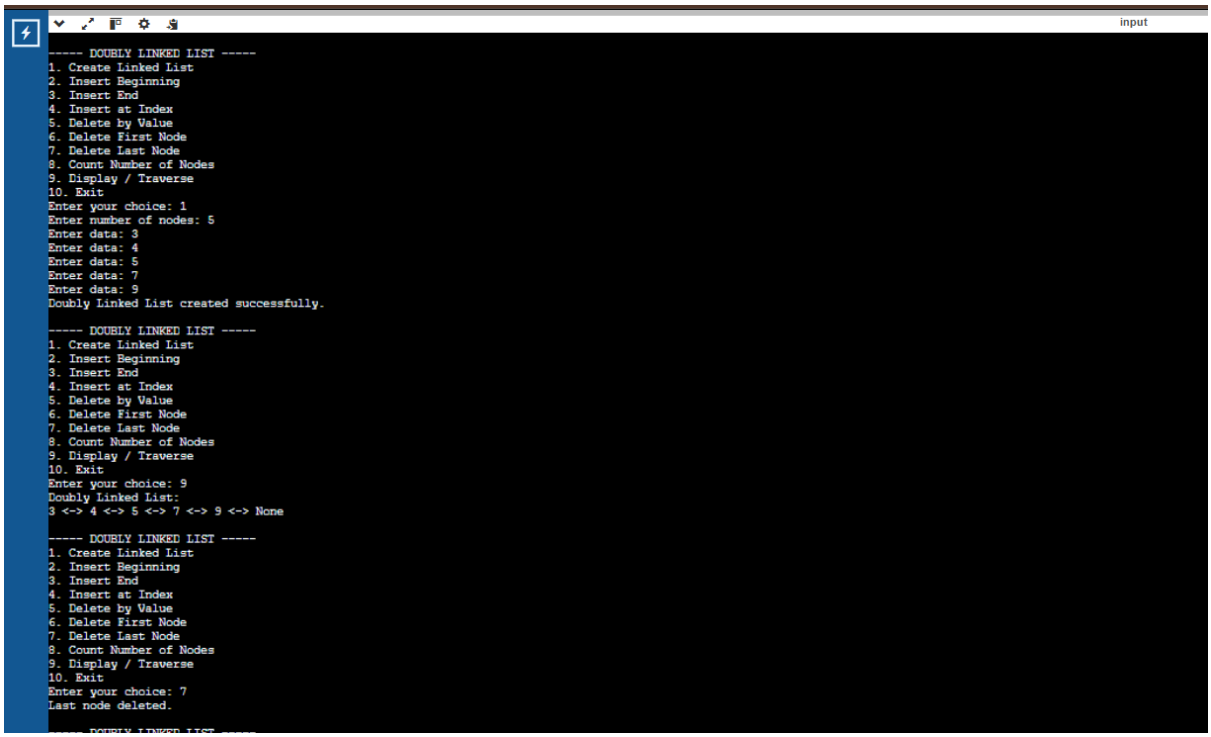
    elif choice == 8:
        dll.count_nodes()

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:



```
input

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data: 3
Enter data: 4
Enter data: 5
Enter data: 7
Enter data: 9
Doubly Linked List created successfully.

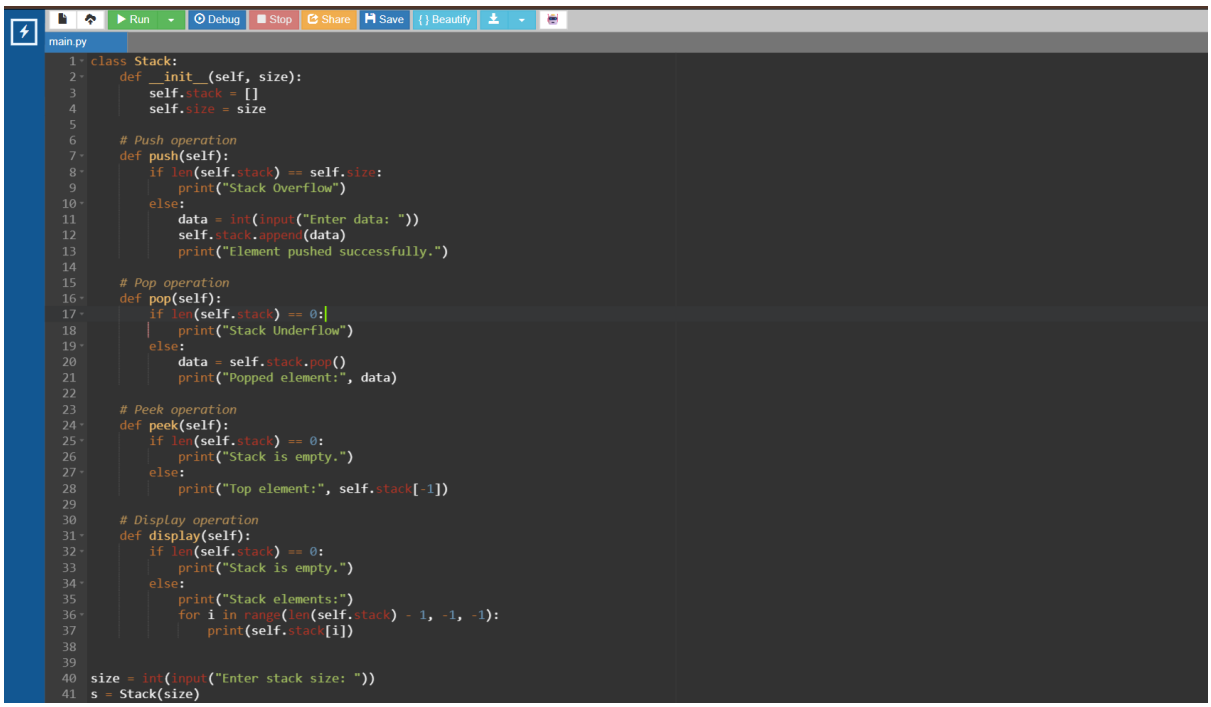
----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
Doubly Linked List:
3 <-> 4 <-> 5 <-> 7 <-> 9 <-> None

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted.
```

WEEK-7:

7(A). Write a python program to implement stack operations such as push, pop, peek, display using an array.

PROGRAM:



Run | Debug | Stop | Share | Save | { } Beautify

main.py

```python
class Stack:
    def __init__(self, size):
        self.stack = []
        self.size = size

    # Push operation
    def push(self):
        if len(self.stack) == self.size:
            print("Stack Overflow")
        else:
            data = int(input("Enter data: "))
            self.stack.append(data)
            print("Element pushed successfully.")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow")
        else:
            data = self.stack.pop()
            print("Popped element:", data)

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element:", self.stack[-1])

    # Display operation
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Stack elements:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])


size = int(input("Enter stack size: "))
s = Stack(size)
```

```python

size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n----- STACK USING ARRAY -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        s.push()

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
input
Enter stack size: 4

----- STACK USING ARRAY -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data: 2
Element pushed successfully.

----- STACK USING ARRAY -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data: 3
Element pushed successfully.

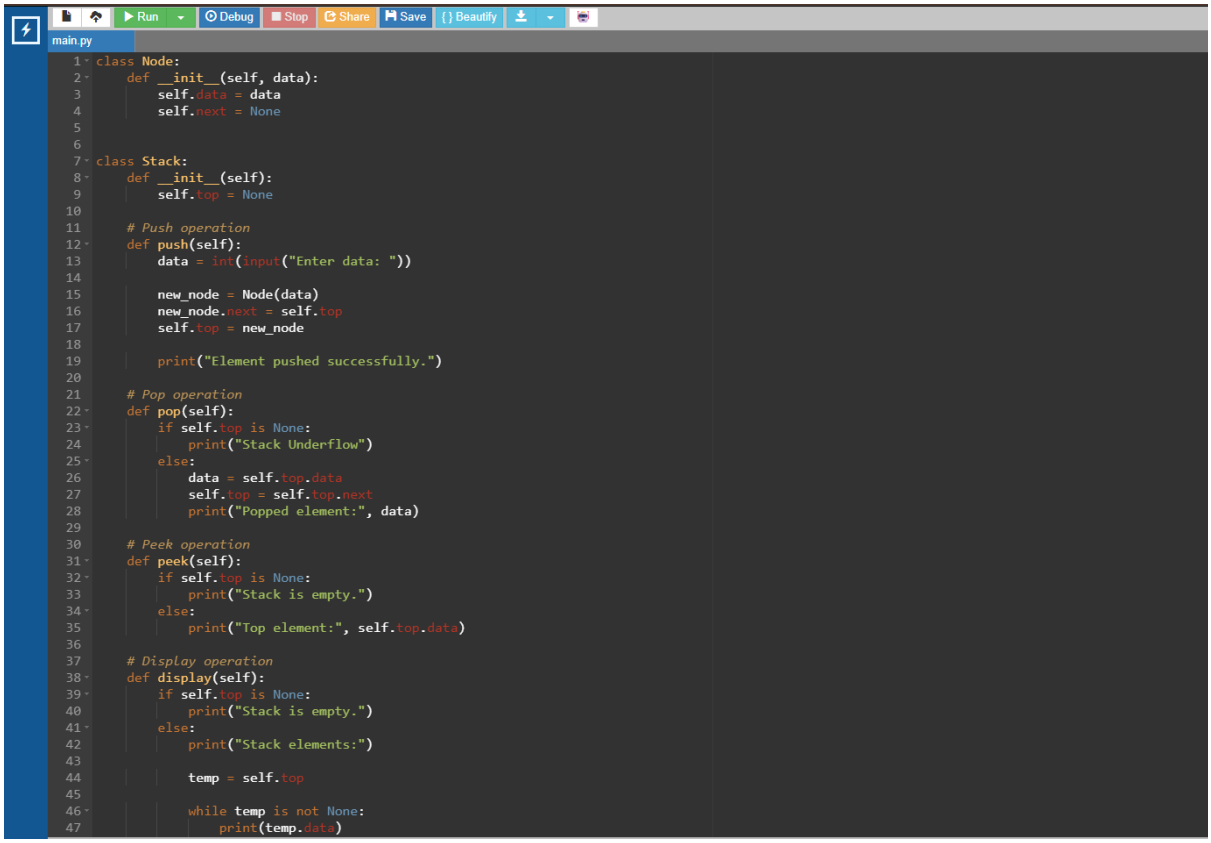
----- STACK USING ARRAY -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data: 5
Element pushed successfully.

----- STACK USING ARRAY -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data: 5
Element pushed successfully.

----- STACK USING ARRAY -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 5
```

7(B). Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

PROGRAM:




```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self):
        data = int(input("Enter data: "))

        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node

        print("Element pushed successfully.")

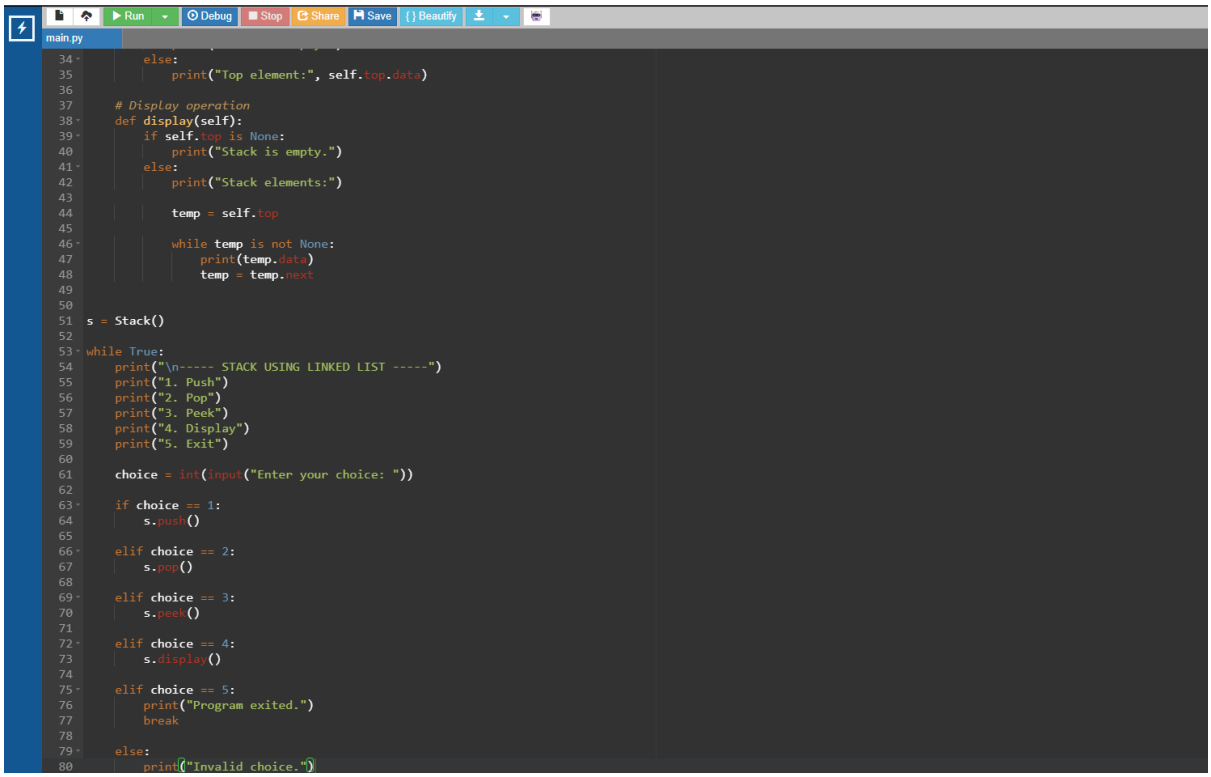
    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            data = self.top.data
            self.top = self.top.next
            print("Popped element:", data)

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty.")
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty.")
        else:
            print("Stack elements:")

            temp = self.top

            while temp is not None:
                print(temp.data)
```




```python
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty.")
        else:
            print("Stack elements:")

            temp = self.top

            while temp is not None:
                print(temp.data)
                temp = temp.next


s = Stack()

while True:
    print("\n----- STACK USING LINKED LIST -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        s.push()

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice.")
```

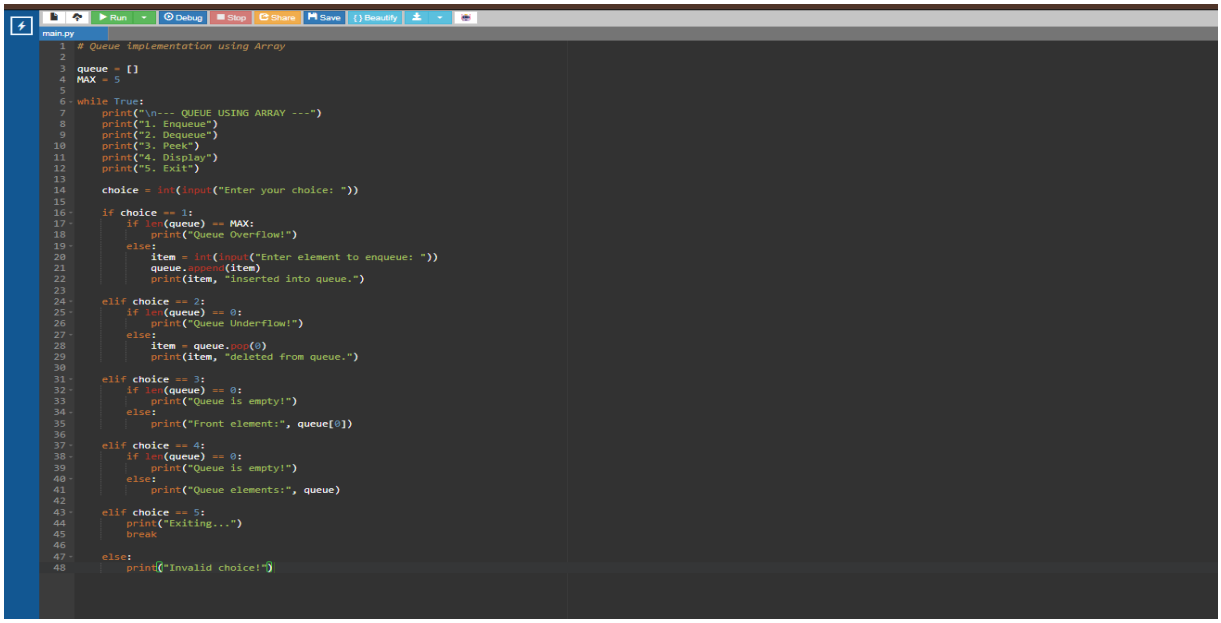
OUTPUT:



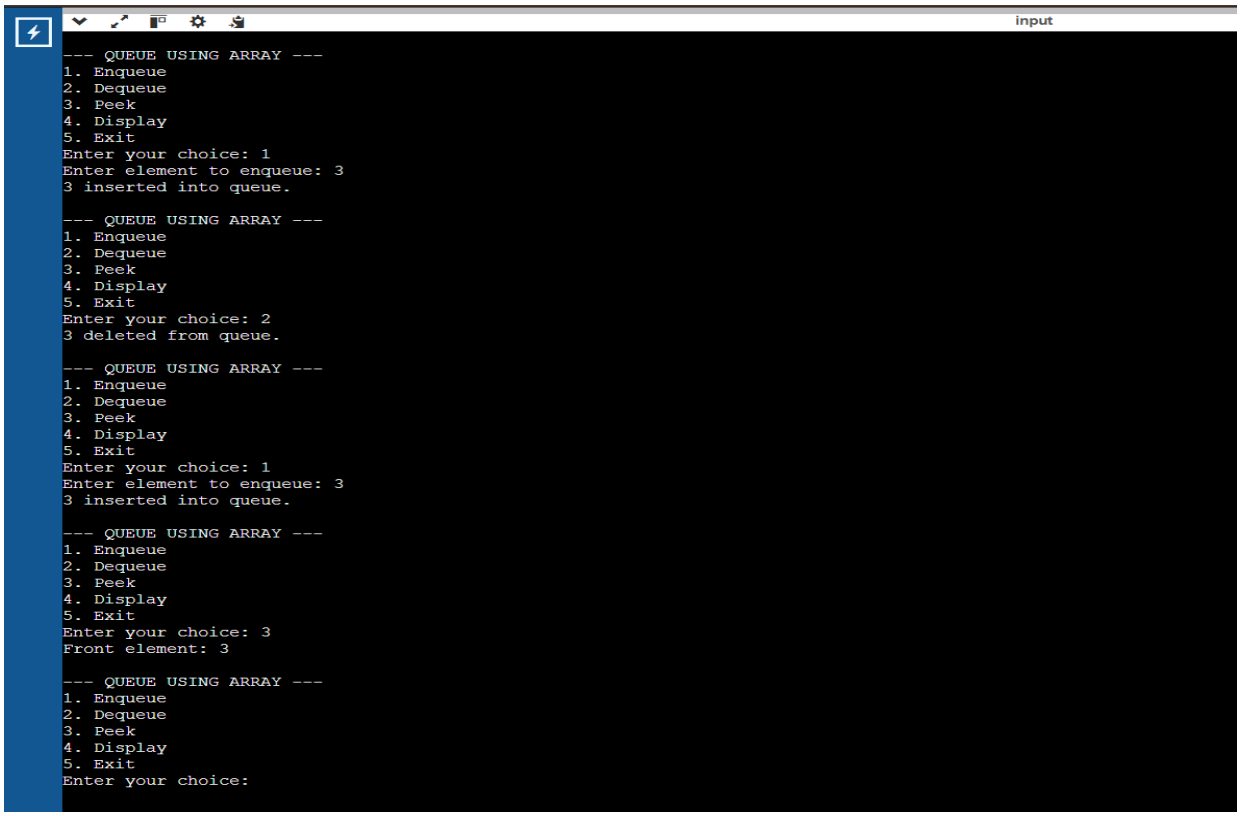
WEEK-8:

8(A). Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

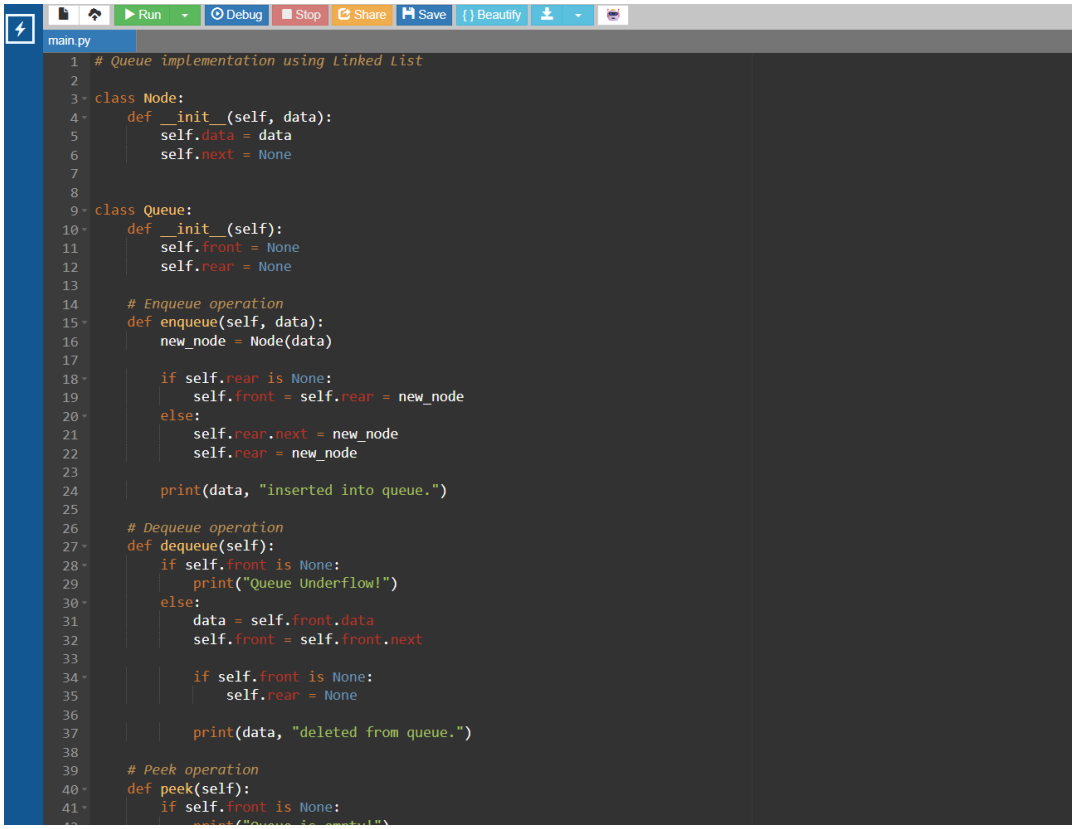
PROGRAM:

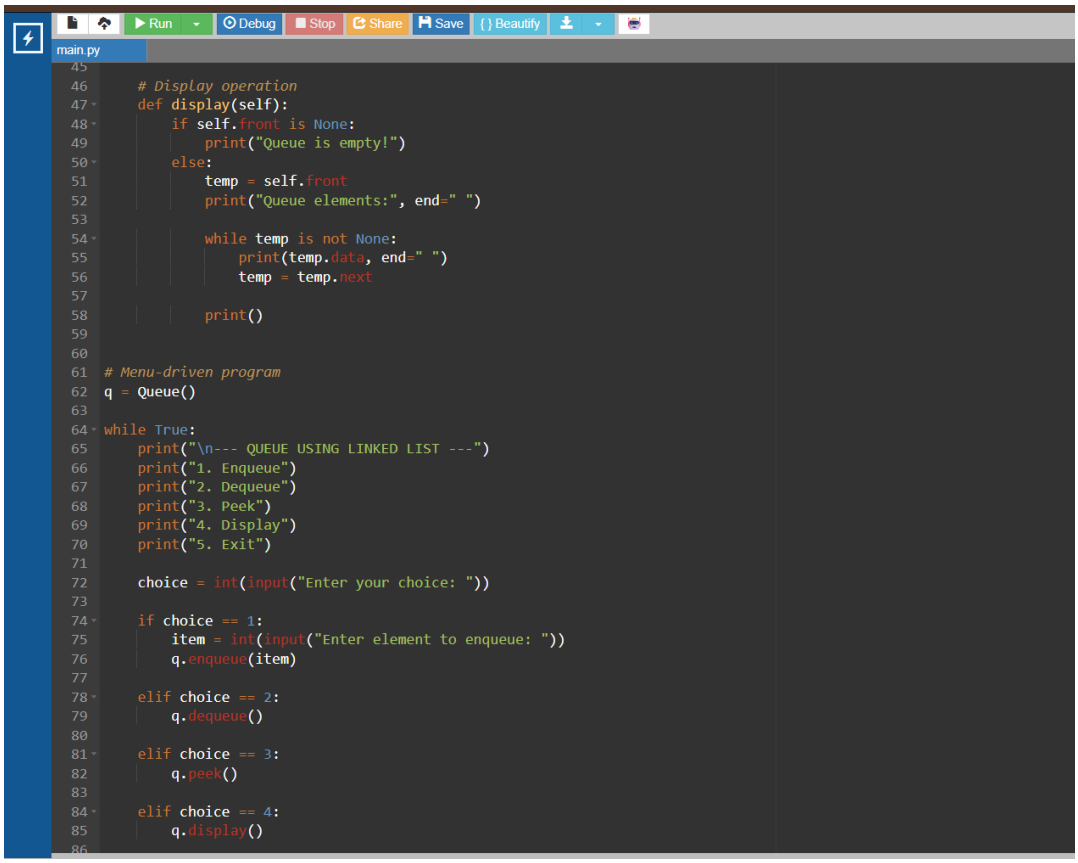


OUTPUT:



LINKED LIST:






```
    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is empty!")
        else:
            temp = self.front
            print("Queue elements:", end=" ")

            while temp is not None:
                print(temp.data, end=" ")
                temp = temp.next

            print()


# Menu-driven program
q = Queue()

while True:
    print("\n--- QUEUE USING LINKED LIST ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()
```



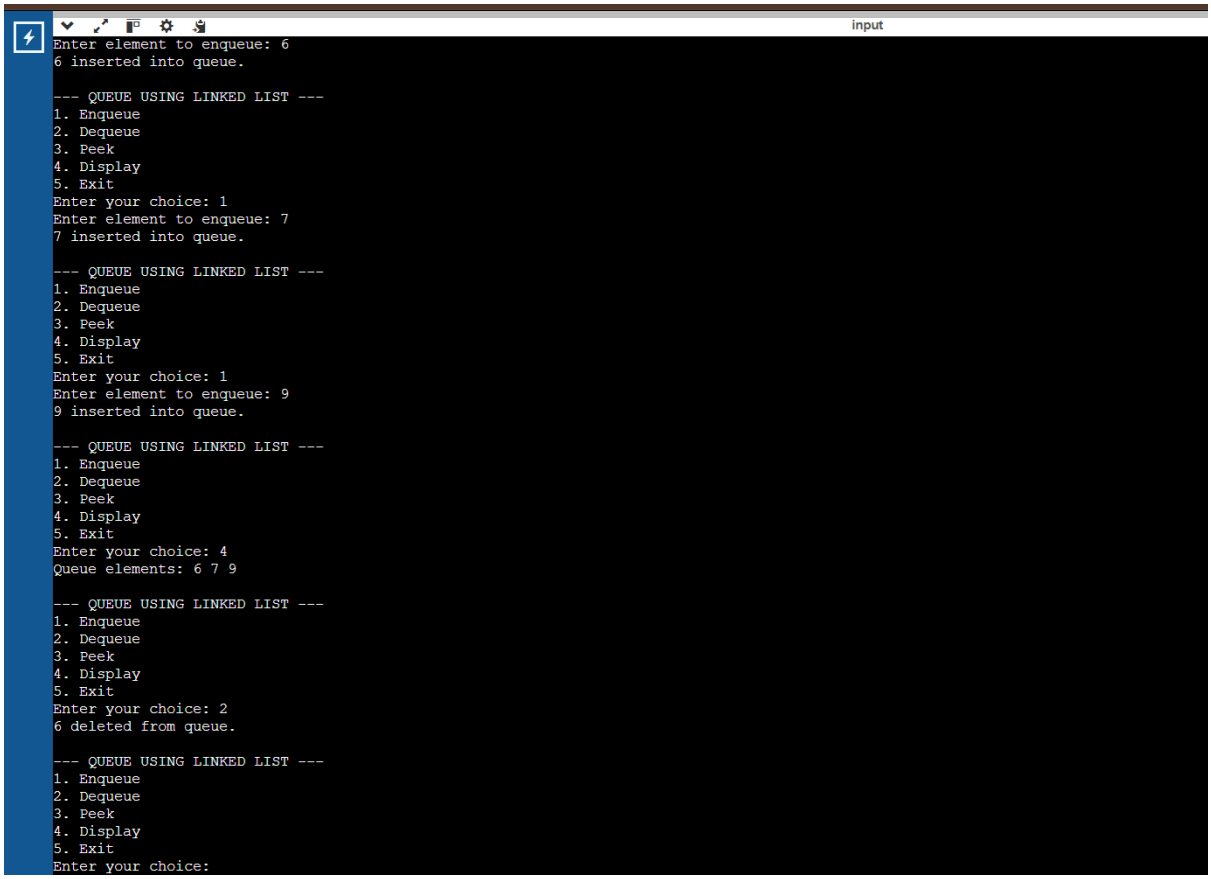
```
    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Invalid choice!")
```

OUTPUT



8(B). Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

PROGRAM:

Run | Debug | Stop | Share | Save | { } Beautify

main.py

```python
# Circular Queue using Array

MAX = 5
queue = [None] * MAX
front = -1
rear = -1

while True:
    print("\n--- CIRCULAR QUEUE USING ARRAY ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    # Enqueue
    if choice == 1:
        if (rear + 1) % MAX == front:
            print("Queue Overflow!")
        else:
            item = int(input("Enter element to enqueue: "))

            if front == -1:
                front = 0

            rear = (rear + 1) % MAX
            queue[rear] = item

            print(item, "inserted into queue.")

    # Dequeue
    elif choice == 2:
        if front == -1:
            print("Queue Underflow!")
        else:
            item = queue[front]
            print(item, "deleted from queue.")
```

```python
            print("Queue Underflow!")
        else:
            item = queue[front]
            print(item, "deleted from queue.")

            if front == rear:
                front = rear = -1
            else:
                front = (front + 1) % MAX

    # Peek
    elif choice == 3:
        if front == -1:
            print("Queue is empty!")
        else:
            print("Front element:", queue[front])

    # Display
    elif choice == 4:
        if front == -1:
            print("Queue is empty!")
        else:
            print("Queue elements:", end=" ")

            i = front
            while True:
                print(queue[i], end=" ")

                if i == rear:
                    break

                i = (i + 1) % MAX

            print()

    # Exit
    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Invalid choice!")
```

OUTPUT:

```
input
Enter element to enqueue: 3
3 inserted into queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 4
4 inserted into queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 6
6 inserted into queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 3 4 6

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
3 deleted from queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

LINKED LIST:

PROGRAM:

Run | Debug | Stop | Share | Save | {} Beautify

main.py

```python
# Circular Queue using Linked List

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, data):
        new_node = Node(data)

        if self.front is None:
            self.front = self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node

        print(data, "inserted into queue.")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow!")
        elif self.front == self.rear:
            data = self.front.data
            self.front = self.rear = None
            print(data, "deleted from queue.")
        else:
            data = self.front.data
            self.front = self.front.next
            self.rear.next = self.front
            print(data, "deleted from queue.")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty!")
        else:
            print("Front element:", self.front.data)
```

```
            print("Front element:", self.front.data)

    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is empty!")
        else:
            print("Queue elements:", end=" ")

            temp = self.front

            while True:
                print(temp.data, end=" ")
                temp = temp.next

                if temp == self.front:
                    break

            print()


# Menu-driven program
q = CircularQueue()

while True:
    print("\n--- CIRCULAR QUEUE USING LINKED LIST ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting...")
        break

    else:
        print("Invalid choice!")
```

OUTPUT:

```
input
Enter your choice: 1
Enter element to enqueue: 4
4 inserted into queue.

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 7
7 inserted into queue.

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 19
Invalid choice!

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 4 7

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 4 7

--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
4 deleted from queue.

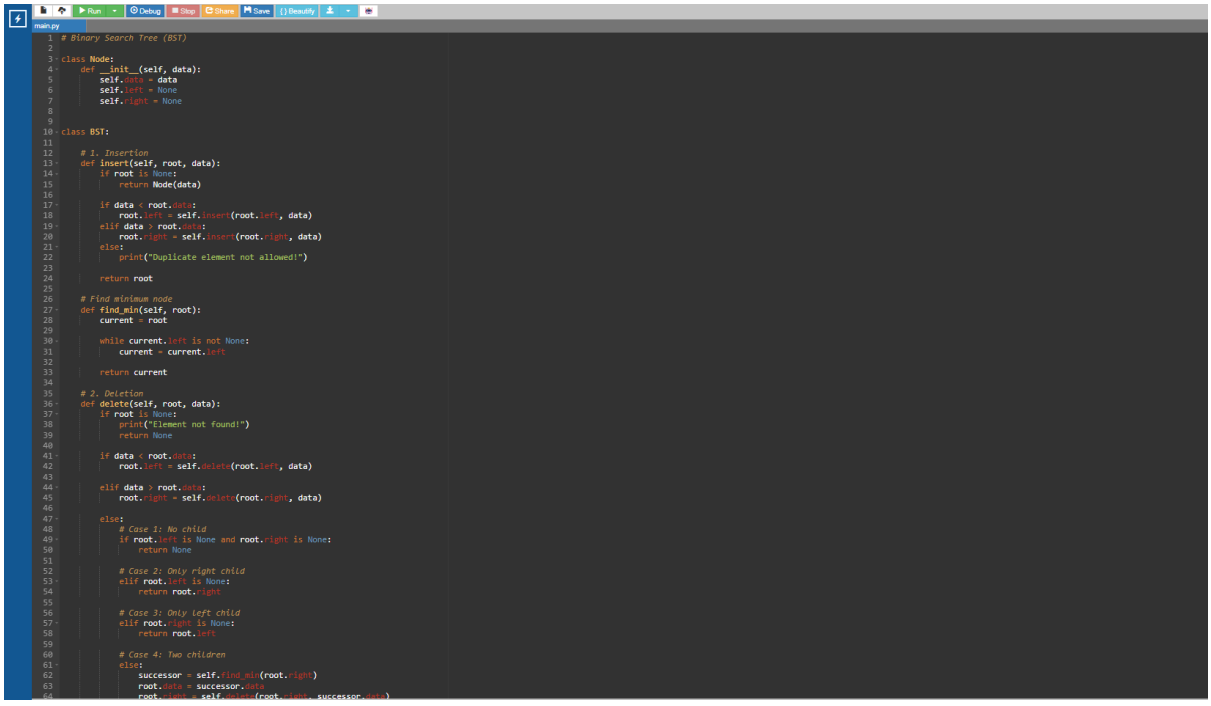
--- CIRCULAR QUEUE USING LINKED LIST ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

WEEK-9:

Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals – in-order, pre-order and post-order
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

PROGRAM:



```
# Binary Search Tree (BST)

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:

    # 1. Insertion
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)
        else:
            print("Duplicate element not allowed!")

        return root

    # Find minimum node
    def find_min(self, root):
        current = root

        while current.left is not None:
            current = current.left

        return current

    # 2. Deletion
    def delete(self, root, data):
        if root is None:
            print("Element not found!")
            return None

        if data < root.data:
            root.left = self.delete(root.left, data)

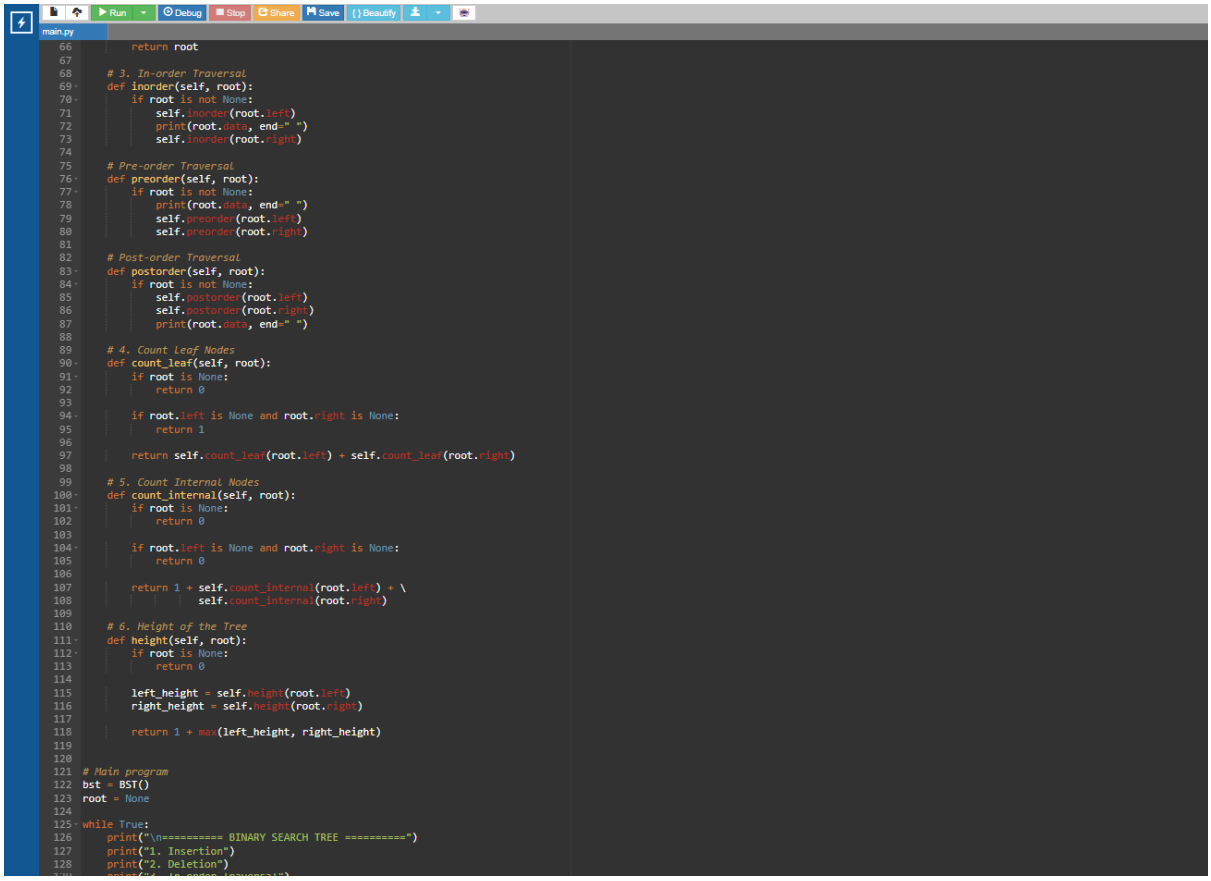
        elif data > root.data:
            root.right = self.delete(root.right, data)

        else:
            # Case 1: No child
            if root.left is None and root.right is None:
                return None

            # Case 2: Only right child
            elif root.left is None:
                return root.right

            # Case 3: Only left child
            elif root.right is None:
                return root.left

            # Case 4: Two children
            else:
                successor = self.find_min(root.right)
                root.data = successor.data
                root.right = self.delete(root.right, successor.data)
```



```
        return root

    # 3. In-order Traversal
    def inorder(self, root):
        if root is not None:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Pre-order Traversal
    def preorder(self, root):
        if root is not None:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Post-order Traversal
    def postorder(self, root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

    # 4. Count Leaf Nodes
    def count_leaf(self, root):
        if root is None:
            return 0

        if root.left is None and root.right is None:
            return 1

        return self.count_leaf(root.left) + self.count_leaf(root.right)

    # 5. Count Internal Nodes
    def count_internal(self, root):
        if root is None:
            return 0

        if root.left is None and root.right is None:
            return 0

        return 1 + self.count_internal(root.left) + \
                   self.count_internal(root.right)

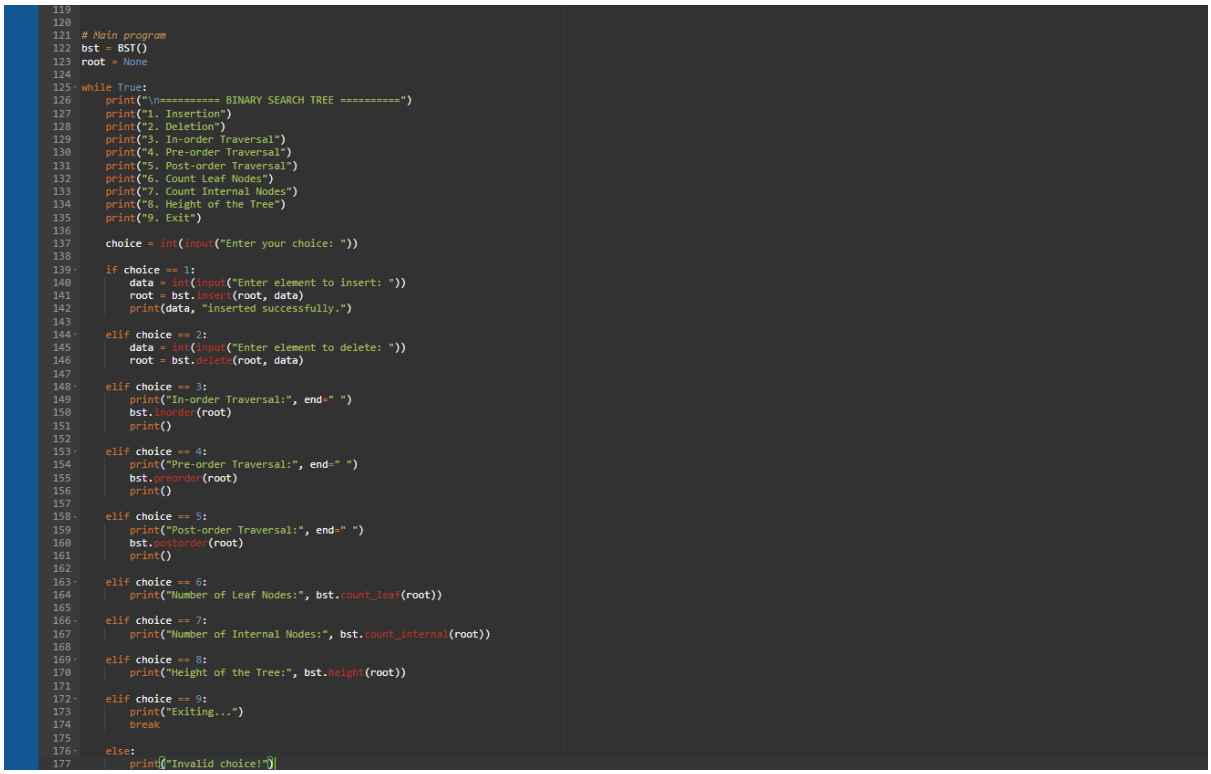
    # 6. Height of the Tree
    def height(self, root):
        if root is None:
            return 0

        left_height = self.height(root.left)
        right_height = self.height(root.right)

        return 1 + max(left_height, right_height)


# Main program
bst = BST()
root = None

while True:
    print("\n========== BINARY SEARCH TREE ==========")
    print("1. Insertion")
    print("2. Deletion")
```



```

# Main program
bst = BST()
root = None

while True:
    print("\n========== BINARY SEARCH TREE ==========")
    print("1. Insertion")
    print("2. Deletion")
    print("3. In-order Traversal")
    print("4. Pre-order Traversal")
    print("5. Post-order Traversal")
    print("6. Count Leaf Nodes")
    print("7. Count Internal Nodes")
    print("8. Height of the Tree")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter element to insert: "))
        root = bst.insert(root, data)
        print(data, "inserted successfully.")

    elif choice == 2:
        data = int(input("Enter element to delete: "))
        root = bst.delete(root, data)

    elif choice == 3:
        print("In-order Traversal:", end=" ")
        bst.inorder(root)
        print()

    elif choice == 4:
        print("Pre-order Traversal:", end=" ")
        bst.preorder(root)
        print()

    elif choice == 5:
        print("Post-order Traversal:", end=" ")
        bst.postorder(root)
        print()

    elif choice == 6:
        print("Number of Leaf Nodes:", bst.count_leaf(root))

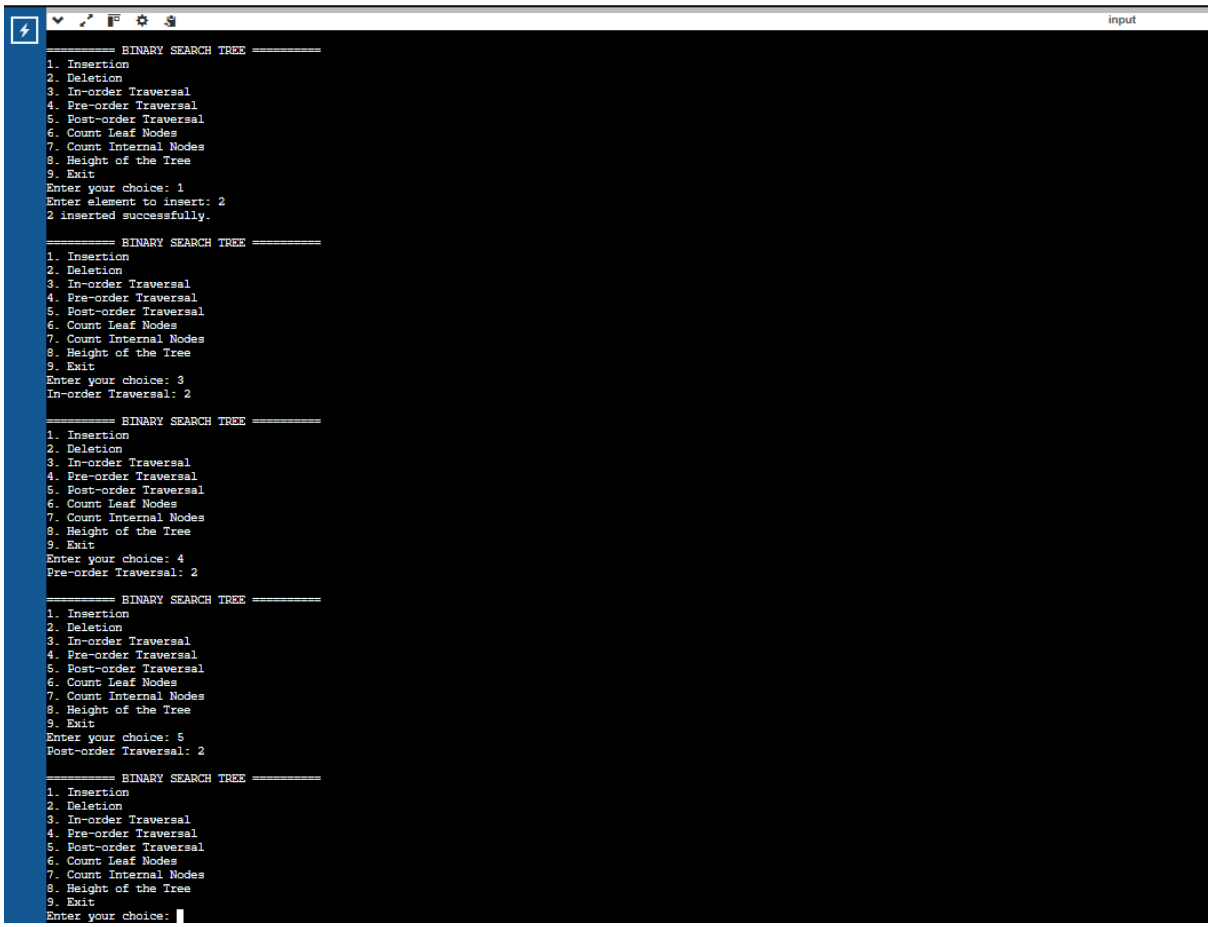
    elif choice == 7:
        print("Number of Internal Nodes:", bst.count_internal(root))

    elif choice == 8:
        print("Height of the Tree:", bst.height(root))

    elif choice == 9:
        print("Exiting...")
        break

    else:
        print("Invalid choice!")
```

OUTPUT:

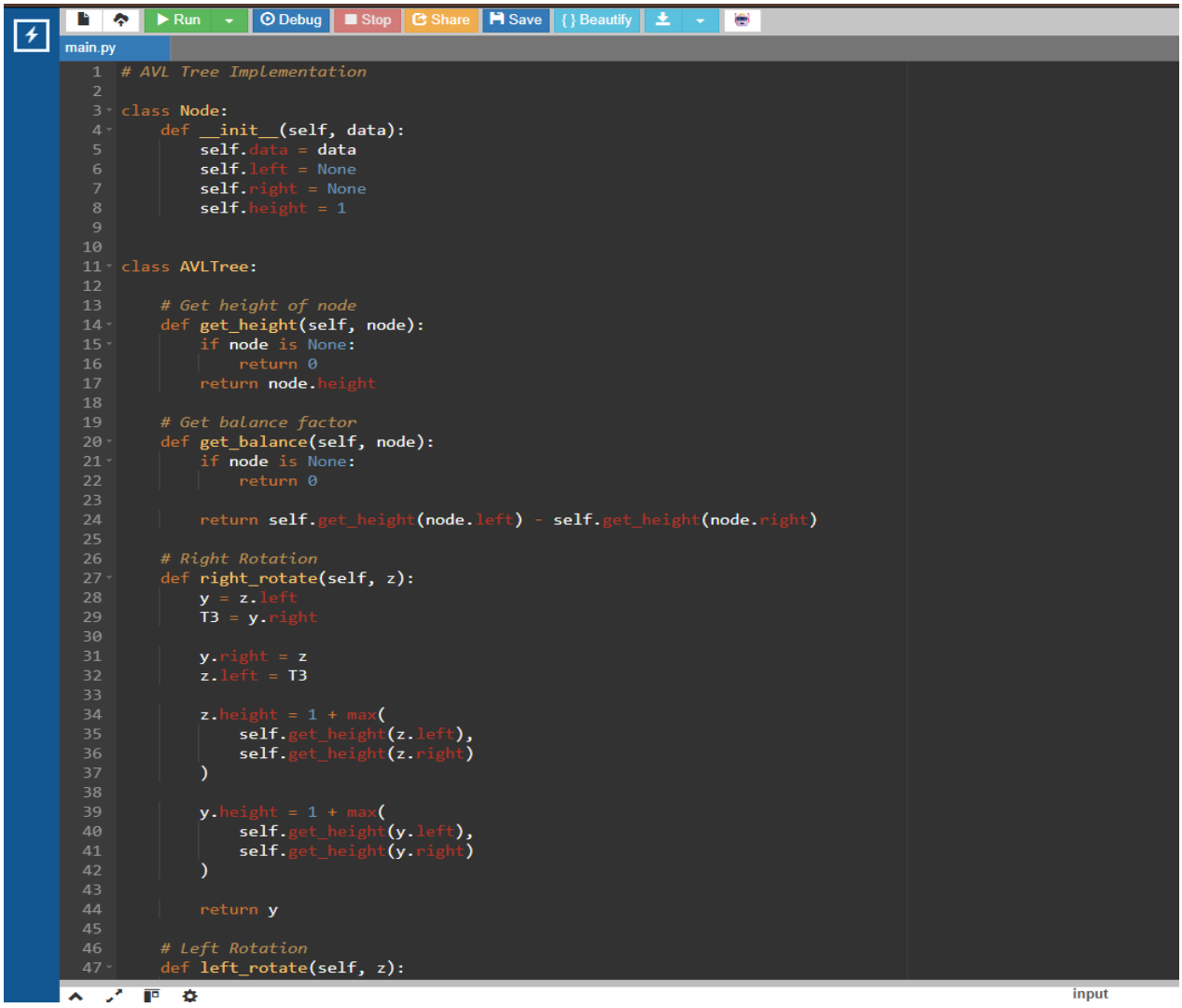

input
========== BINARY SEARCH TREE ==========
1. Insertion
2. Deletion
3. In-order Traversal
4. Pre-order Traversal
5. Post-order Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Height of the Tree
9. Exit
Enter your choice: 1
Enter element to insert: 2
2 inserted successfully.
========== BINARY SEARCH TREE ==========
1. Insertion
2. Deletion
3. In-order Traversal
4. Pre-order Traversal
5. Post-order Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Height of the Tree
9. Exit
Enter your choice: 3
In-order Traversal: 2
========== BINARY SEARCH TREE ==========
1. Insertion
2. Deletion
3. In-order Traversal
4. Pre-order Traversal
5. Post-order Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Height of the Tree
9. Exit
Enter your choice: 4
Pre-order Traversal: 2
========== BINARY SEARCH TREE ==========
1. Insertion
2. Deletion
3. In-order Traversal
4. Pre-order Traversal
5. Post-order Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Height of the Tree
9. Exit
Enter your choice: 5
Post-order Traversal: 2
========== BINARY SEARCH TREE ==========
1. Insertion
2. Deletion
3. In-order Traversal
4. Pre-order Traversal
5. Post-order Traversal
6. Count Leaf Nodes
7. Count Internal Nodes
8. Height of the Tree
9. Exit
Enter your choice:

WEEK-10:

Write a menu driven python program to implement an AVL Tree and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

PROGRAM:




```python
# AVL Tree Implementation

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:

    # Get height of node
    def get_height(self, node):
        if node is None:
            return 0
        return node.height

    # Get balance factor
    def get_balance(self, node):
        if node is None:
            return 0

        return self.get_height(node.left) - self.get_height(node.right)

    # Right Rotation
    def right_rotate(self, z):
        y = z.left
        T3 = y.right

        y.right = z
        z.left = T3

        z.height = 1 + max(
            self.get_height(z.left),
            self.get_height(z.right)
        )

        y.height = 1 + max(
            self.get_height(y.left),
            self.get_height(y.right)
        )

        return y

    # Left Rotation
    def left_rotate(self, z):
```

Run | Debug | Stop | Share | Save | { } Beautify

main.py

```
    # Left Rotation
    def left_rotate(self, z):
        y = z.right
        T2 = y.left

        y.left = z
        z.right = T2

        z.height = 1 + max(
            self.get_height(z.left),
            self.get_height(z.right)
        )

        y.height = 1 + max(
            self.get_height(y.left),
            self.get_height(y.right)
        )

        return y

    # 1. Insertion
    def insert(self, root, data):

        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)

        elif data > root.data:
            root.right = self.insert(root.right, data)

        else:
            print("Duplicate element not allowed!")
            return root

        # Update height
        root.height = 1 + max(
            self.get_height(root.left),
            self.get_height(root.right)
        )

        # Get balance factor
        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and data < root.left.data:
```

Run | Debug | Stop | Share | Save | { } Beautify

main.py

```
        # LR Case
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # 3. In-order Traversal
    def inorder(self, root):
        if root is not None:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

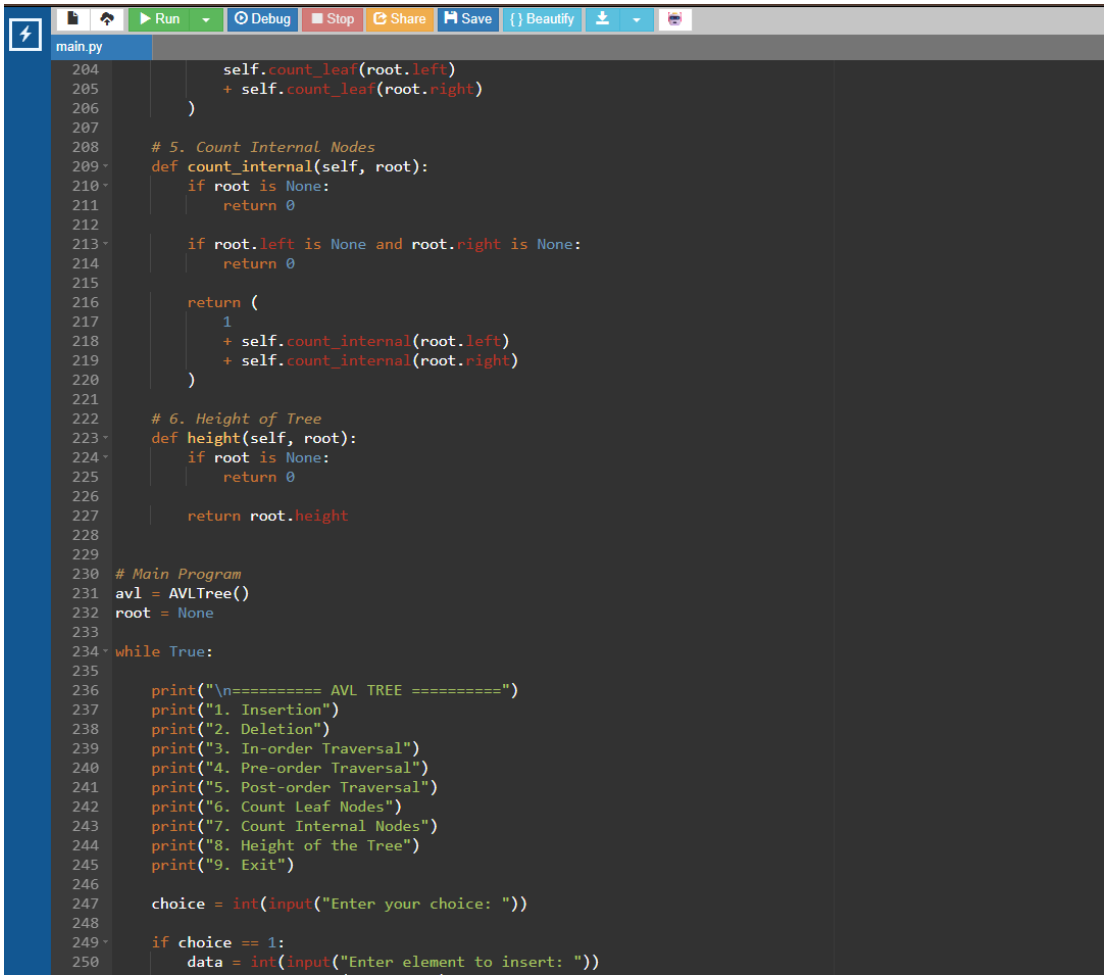
    # Pre-order Traversal
    def preorder(self, root):
        if root is not None:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Post-order Traversal
    def postorder(self, root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

    # 4. Count Leaf Nodes
    def count_leaf(self, root):
        if root is None:
            return 0

        if root.left is None and root.right is None:
            return 1

        return (
```



```
            self.count_leaf(root.left)
            + self.count_leaf(root.right)
        )

    # 5. Count Internal Nodes
    def count_internal(self, root):
        if root is None:
            return 0

        if root.left is None and root.right is None:
            return 0

        return (
            1
            + self.count_internal(root.left)
            + self.count_internal(root.right)
        )

    # 6. Height of Tree
    def height(self, root):
        if root is None:
            return 0

        return root.height

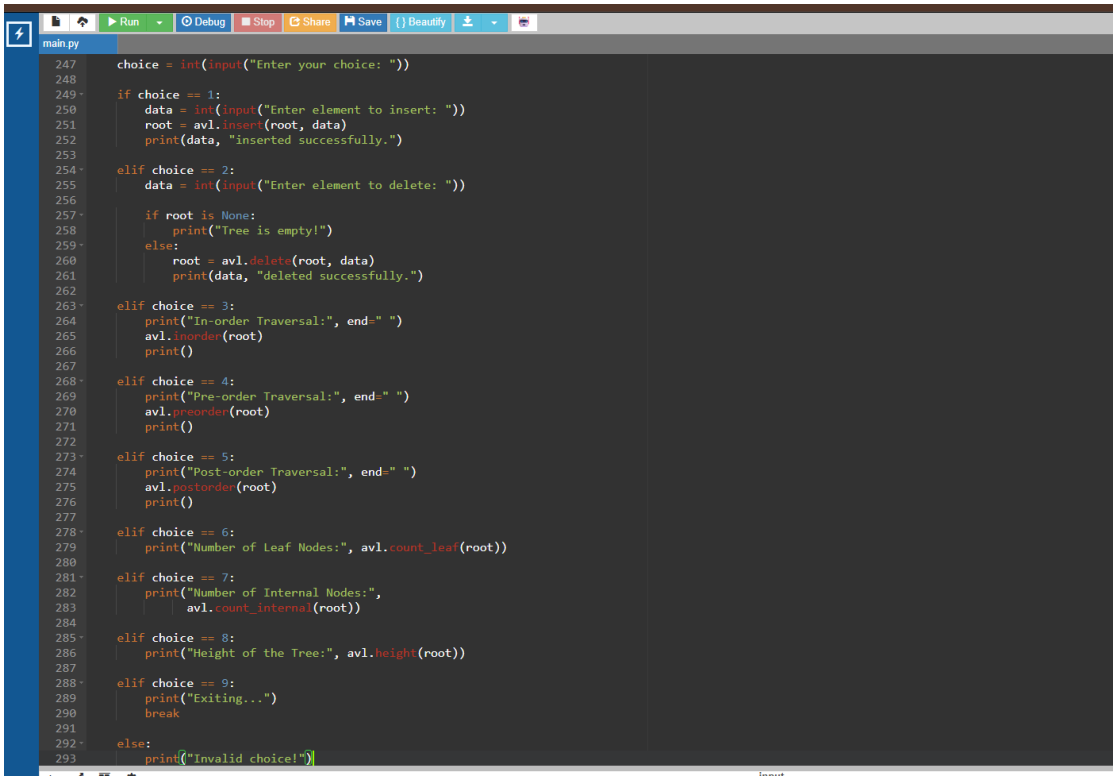

# Main Program
avl = AVLTree()
root = None

while True:

    print("\n========== AVL TREE ==========")
    print("1. Insertion")
    print("2. Deletion")
    print("3. In-order Traversal")
    print("4. Pre-order Traversal")
    print("5. Post-order Traversal")
    print("6. Count Leaf Nodes")
    print("7. Count Internal Nodes")
    print("8. Height of the Tree")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter element to insert: "))
```



```
choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter element to insert: "))
    root = avl.insert(root, data)
    print(data, "inserted successfully.")

elif choice == 2:
    data = int(input("Enter element to delete: "))

    if root is None:
        print("Tree is empty!")
    else:
        root = avl.delete(root, data)
        print(data, "deleted successfully.")

elif choice == 3:
    print("In-order Traversal:", end=" ")
    avl.inorder(root)
    print()

elif choice == 4:
    print("Pre-order Traversal:", end=" ")
    avl.preorder(root)
    print()

elif choice == 5:
    print("Post-order Traversal:", end=" ")
    avl.postorder(root)
    print()

elif choice == 6:
    print("Number of Leaf Nodes:", avl.count_leaf(root))

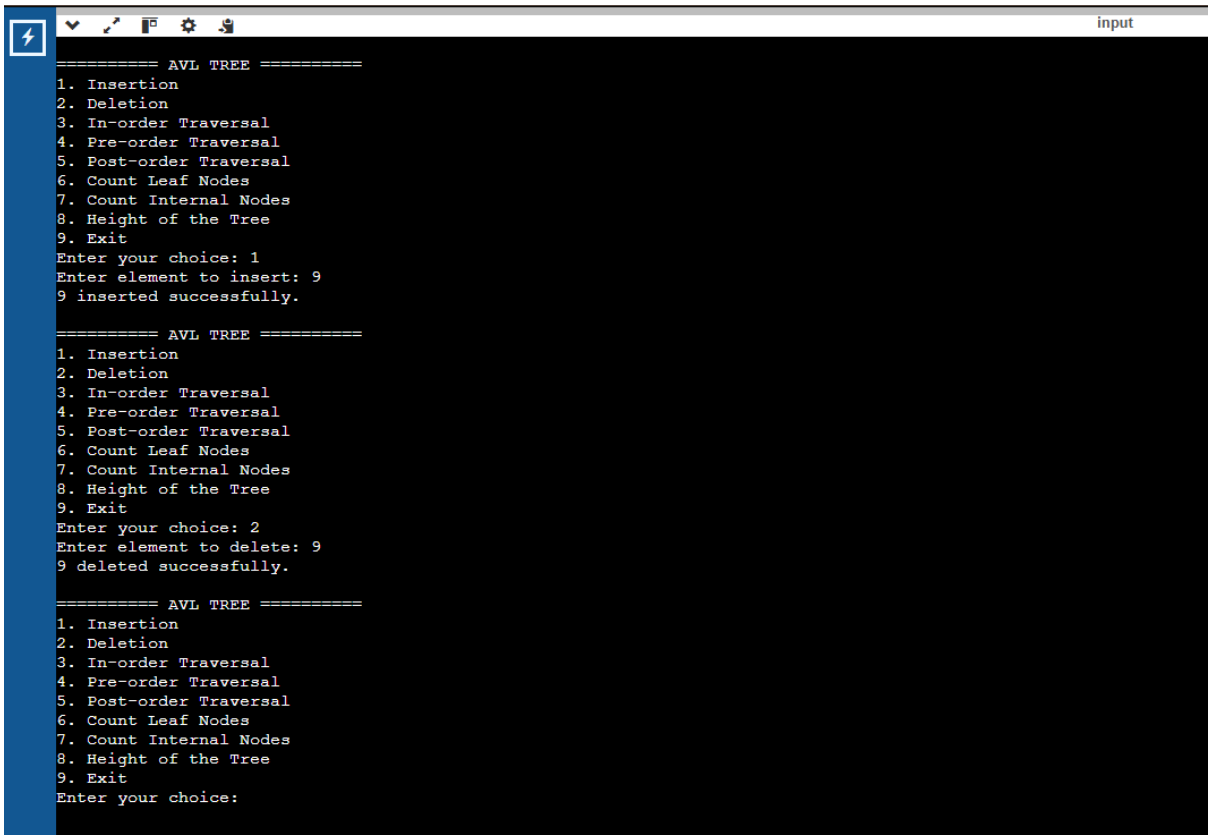
elif choice == 7:
    print("Number of Internal Nodes:",
          avl.count_internal(root))

elif choice == 8:
    print("Height of the Tree:", avl.height(root))

elif choice == 9:
    print("Exiting...")
    break

else:
    print("Invalid choice!")
```

OUTPUT:

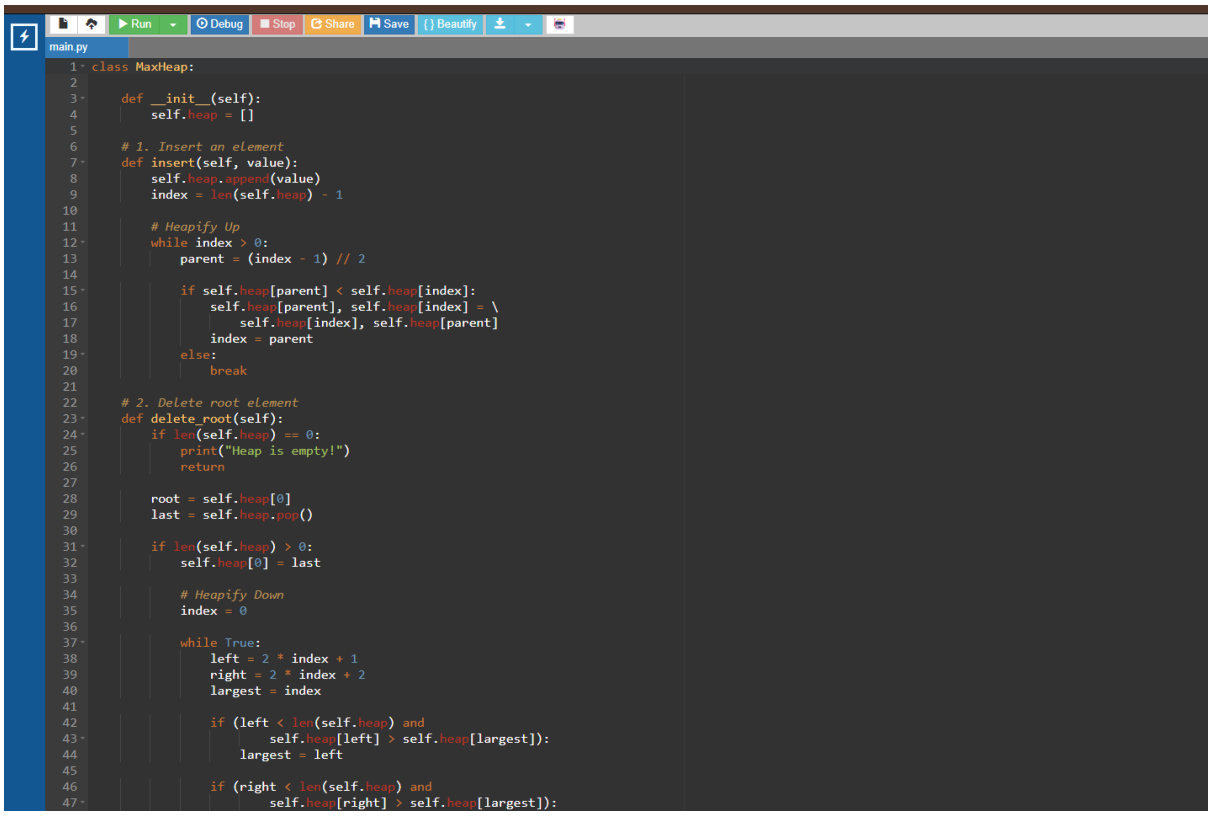


WEEK-11:

Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

1. Insertion of an element into the Heap.
2. Deletion of the root element from the Heap.
3. Display the array representation of the Heap.
4. Exit the program.

PROGRAM:



```
class MaxHeap:

    def __init__(self):
        self.heap = []

    # 1. Insert an element
    def insert(self, value):
        self.heap.append(value)
        index = len(self.heap) - 1

        # Heapify Up
        while index > 0:
            parent = (index - 1) // 2

            if self.heap[parent] < self.heap[index]:
                self.heap[parent], self.heap[index] = \
                    self.heap[index], self.heap[parent]
                index = parent
            else:
                break

    # 2. Delete root element
    def delete_root(self):
        if len(self.heap) == 0:
            print("Heap is empty!")
            return

        root = self.heap[0]
        last = self.heap.pop()

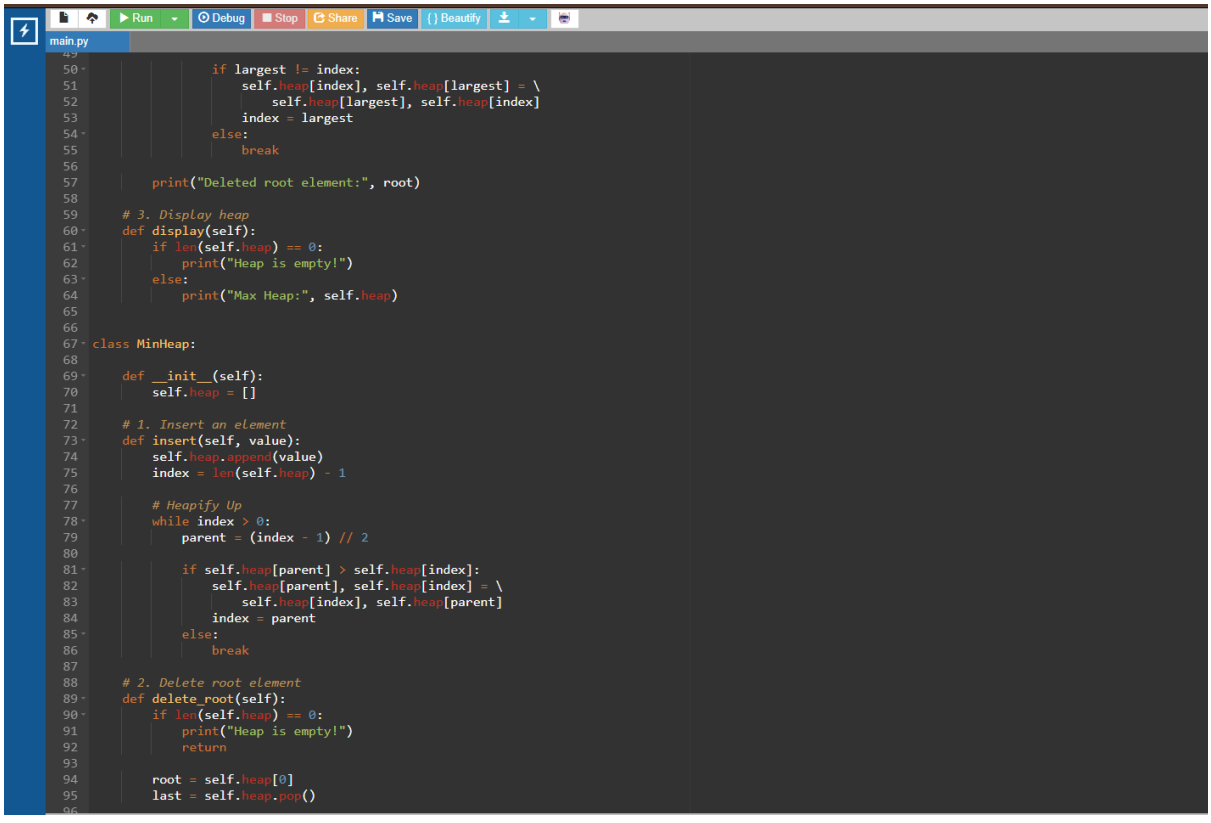
        if len(self.heap) > 0:
            self.heap[0] = last

            # Heapify Down
            index = 0

            while True:
                left = 2 * index + 1
                right = 2 * index + 2
                largest = index

                if (left < len(self.heap) and
                        self.heap[left] > self.heap[largest]):
                    largest = left

                if (right < len(self.heap) and
                        self.heap[right] > self.heap[largest]):
```



```
                if largest != index:
                    self.heap[index], self.heap[largest] = \
                        self.heap[largest], self.heap[index]
                    index = largest
                else:
                    break

        print("Deleted root element:", root)

    # 3. Display heap
    def display(self):
        if len(self.heap) == 0:
            print("Heap is empty!")
        else:
            print("Max Heap:", self.heap)


class MinHeap:

    def __init__(self):
        self.heap = []

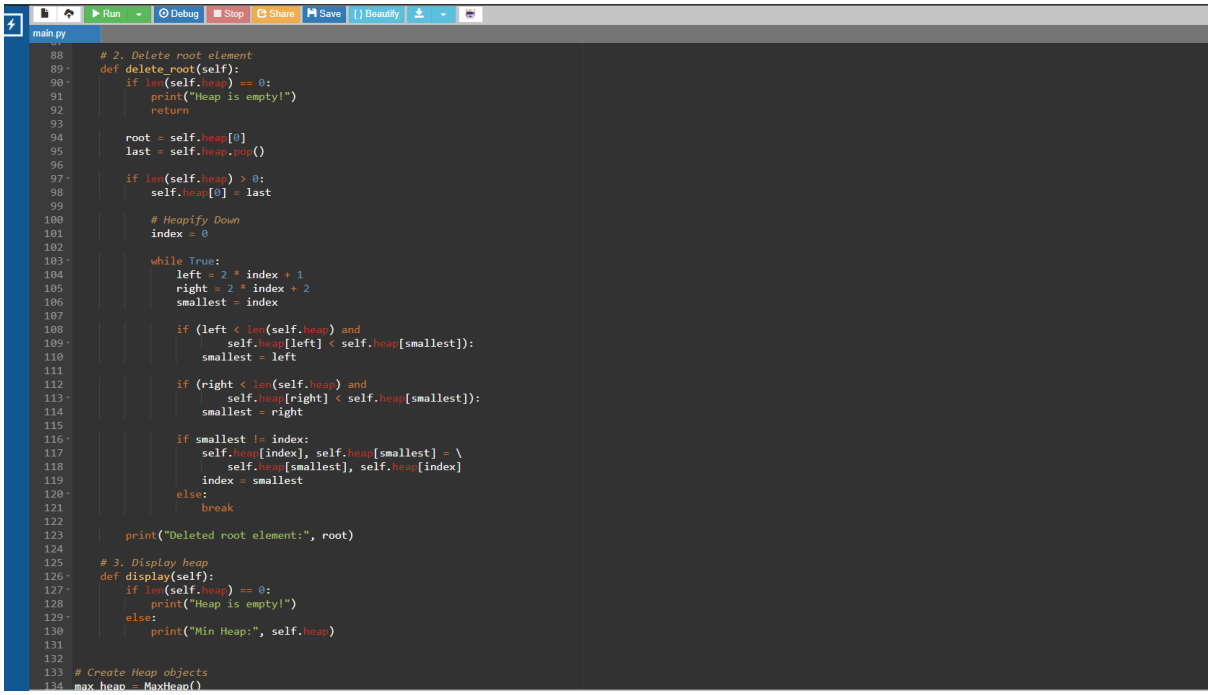
    # 1. Insert an element
    def insert(self, value):
        self.heap.append(value)
        index = len(self.heap) - 1

        # Heapify Up
        while index > 0:
            parent = (index - 1) // 2

            if self.heap[parent] > self.heap[index]:
                self.heap[parent], self.heap[index] = \
                    self.heap[index], self.heap[parent]
                index = parent
            else:
                break

    # 2. Delete root element
    def delete_root(self):
        if len(self.heap) == 0:
            print("Heap is empty!")
            return

        root = self.heap[0]
        last = self.heap.pop()
```



```
    # 2. Delete root element
    def delete_root(self):
        if len(self.heap) == 0:
            print("Heap is empty!")
            return

        root = self.heap[0]
        last = self.heap.pop()

        if len(self.heap) > 0:
            self.heap[0] = last

            # Heapify Down
            index = 0

            while True:
                left = 2 * index + 1
                right = 2 * index + 2
                smallest = index

                if (left < len(self.heap) and
                        self.heap[left] < self.heap[smallest]):
                    smallest = left

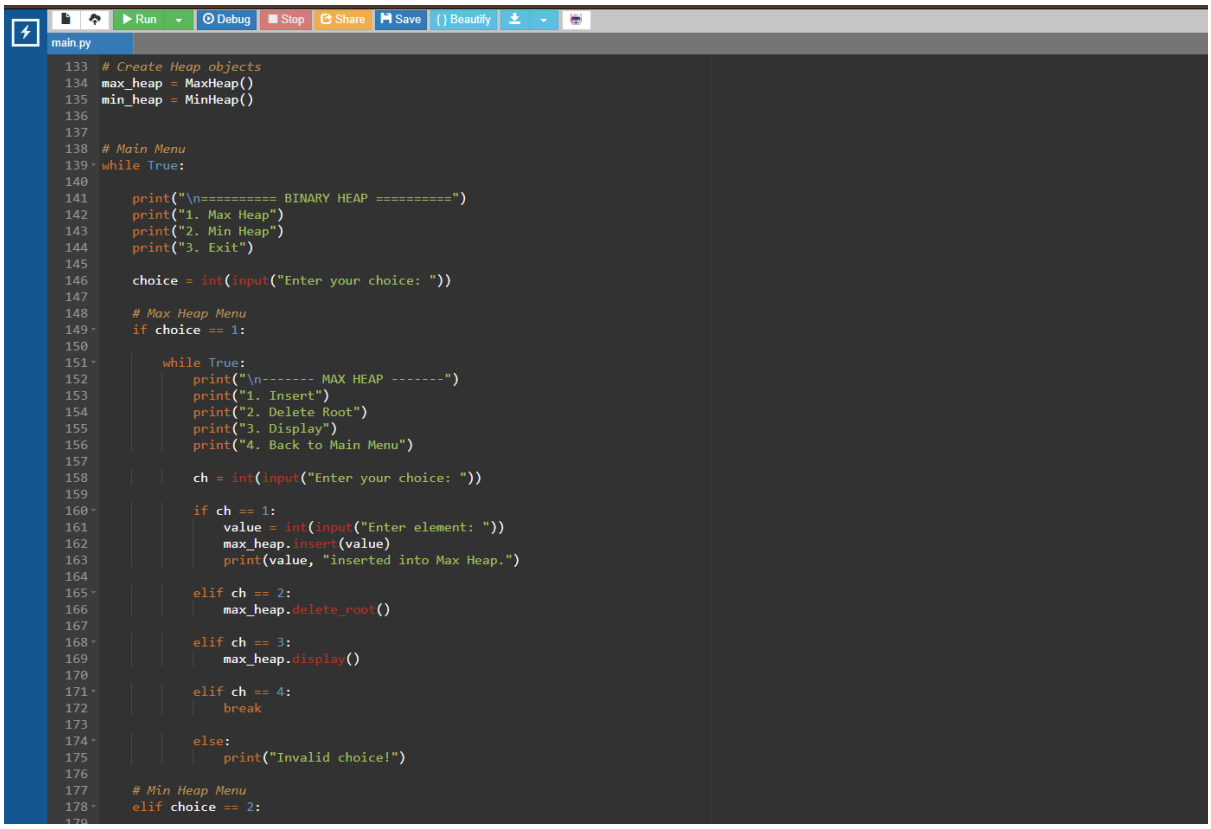
                if (right < len(self.heap) and
                        self.heap[right] < self.heap[smallest]):
                    smallest = right

                if smallest != index:
                    self.heap[index], self.heap[smallest] = \
                        self.heap[smallest], self.heap[index]
                    index = smallest
                else:
                    break

        print("Deleted root element:", root)

    # 3. Display heap
    def display(self):
        if len(self.heap) == 0:
            print("Heap is empty!")
        else:
            print("Min Heap:", self.heap)


# Create Heap objects
max_heap = MaxHeap()
```



```
# Create Heap objects
max_heap = MaxHeap()
min_heap = MinHeap()


# Main Menu
while True:

    print("\n========== BINARY HEAP ==========")
    print("1. Max Heap")
    print("2. Min Heap")
    print("3. Exit")

    choice = int(input("Enter your choice: "))

    # Max Heap Menu
    if choice == 1:

        while True:
            print("\n------- MAX HEAP -------")
            print("1. Insert")
            print("2. Delete Root")
            print("3. Display")
            print("4. Back to Main Menu")

            ch = int(input("Enter your choice: "))

            if ch == 1:
                value = int(input("Enter element: "))
                max_heap.insert(value)
                print(value, "inserted into Max Heap.")

            elif ch == 2:
                max_heap.delete_root()

            elif ch == 3:
                max_heap.display()

            elif ch == 4:
                break

            else:
                print("Invalid choice!")

    # Min Heap Menu
    elif choice == 2:
```

```
176
177     # Min Heap Menu
178     elif choice == 2:
179
180         while True:
181             print("\n------- MIN HEAP -------")
182             print("1. Insert")
183             print("2. Delete Root")
184             print("3. Display")
185             print("4. Back to Main Menu")
186
187             ch = int(input("Enter your choice: "))
188
189             if ch == 1:
190                 value = int(input("Enter element: "))
191                 min_heap.insert(value)
192                 print(value, "inserted into Min Heap.")
193
194             elif ch == 2:
195                 min_heap.delete_root()
196
197             elif ch == 3:
198                 min_heap.display()
199
200             elif ch == 4:
201                 break
202
203             else:
204                 print("Invalid choice!")
205
206     # Exit
207     elif choice == 3:
208         print("Exiting program...")
209         break
210
211     else:
212         print("Invalid choice!")
```

OUTPUT:

```
input
------- MAX HEAP -------
1. Insert
2. Delete Root
3. Display
4. Back to Main Menu
Enter your choice: 1
Enter element: 3
3 inserted into Max Heap.

------- MAX HEAP -------
1. Insert
2. Delete Root
3. Display
4. Back to Main Menu
Enter your choice: 1
Enter element: 4
4 inserted into Max Heap.

------- MAX HEAP -------
1. Insert
2. Delete Root
3. Display
4. Back to Main Menu
Enter your choice: 1
Enter element: 8
8 inserted into Max Heap.

------- MAX HEAP -------
1. Insert
2. Delete Root
3. Display
4. Back to Main Menu
Enter your choice: 1
Enter element: 9
9 inserted into Max Heap.

------- MAX HEAP -------
1. Insert
2. Delete Root
3. Display
4. Back to Main Menu
Enter your choice: 3
Max Heap: [9, 8, 4, 3]

------- MAX HEAP -------
1. Insert
2. Delete Root
3. Display
4. Back to Main Menu
Enter your choice:
```

WEEK-12:

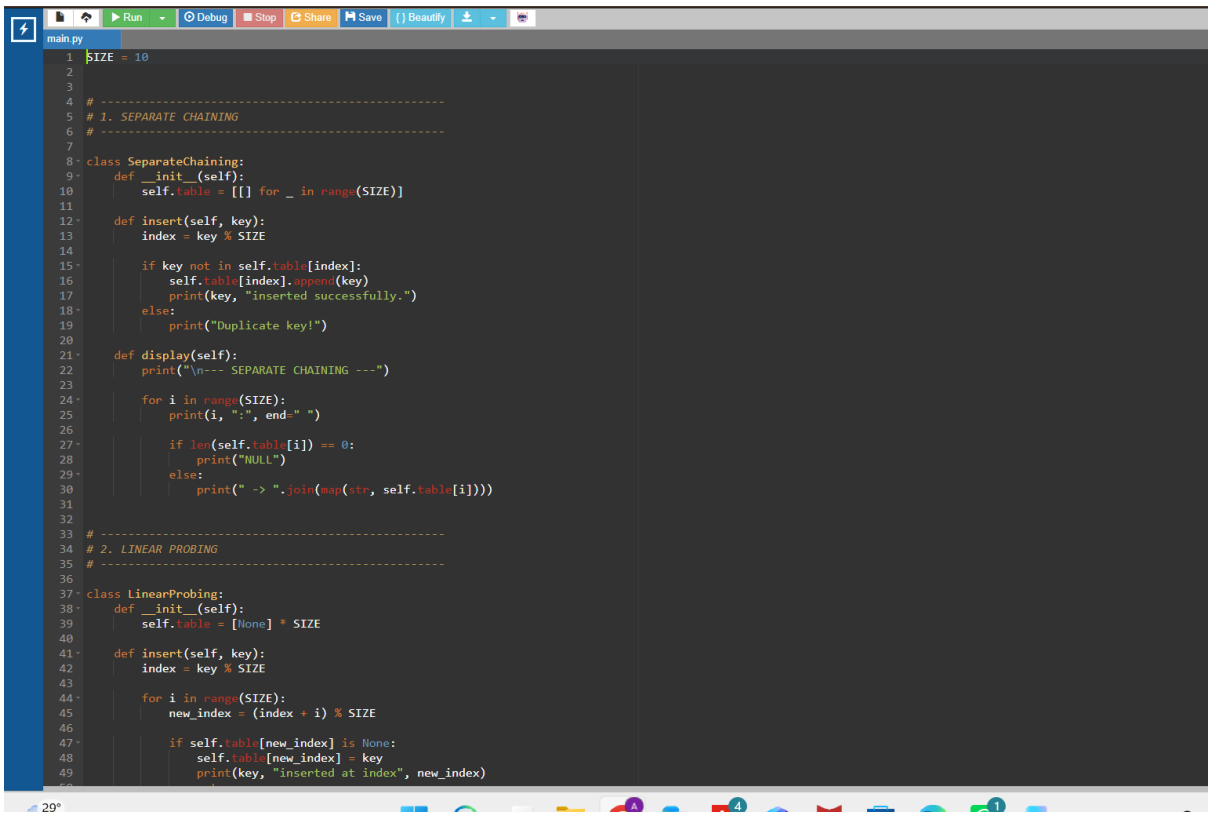
Write a program to implement the following three collision resolutiontechniques in a hash table :

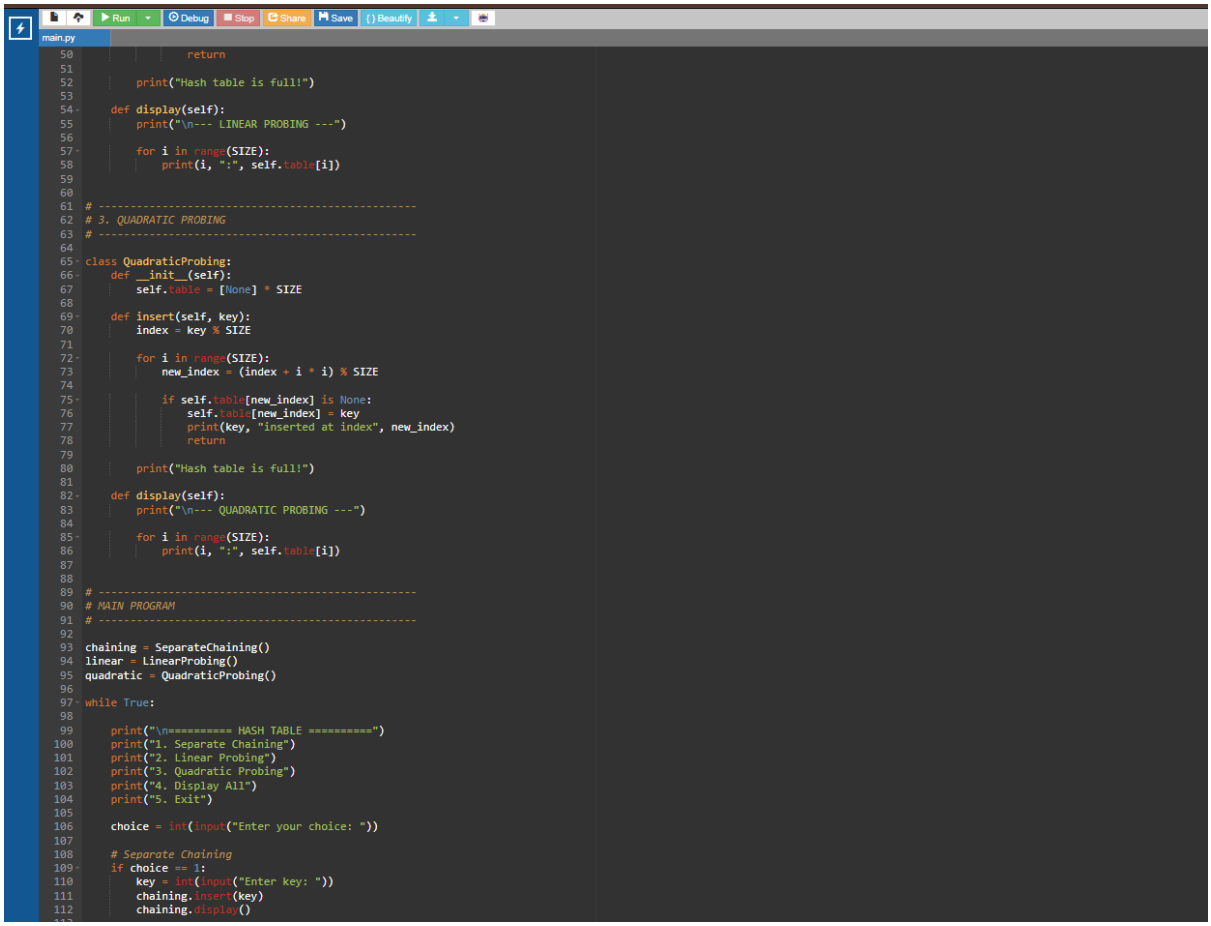
1.separate chaining

2. linear probing

3. quadratic probing

PROGRAM:





```
                return

        print("Hash table is full!")

    def display(self):
        print("\n--- LINEAR PROBING ---")

        for i in range(SIZE):
            print(i, ":", self.table[i])


# ------------------------------------------------
# 3. QUADRATIC PROBING
# ------------------------------------------------

class QuadraticProbing:
    def __init__(self):
        self.table = [None] * SIZE

    def insert(self, key):
        index = key % SIZE

        for i in range(SIZE):
            new_index = (index + i * i) % SIZE

            if self.table[new_index] is None:
                self.table[new_index] = key
                print(key, "inserted at index", new_index)
                return

        print("Hash table is full!")

    def display(self):
        print("\n--- QUADRATIC PROBING ---")

        for i in range(SIZE):
            print(i, ":", self.table[i])


# ------------------------------------------------
# MAIN PROGRAM
# ------------------------------------------------

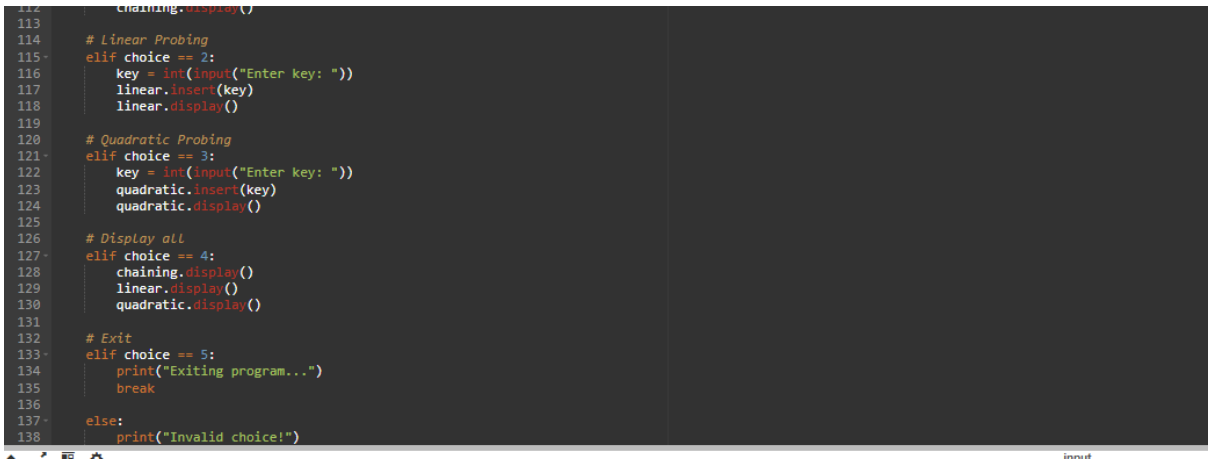
chaining = SeparateChaining()
linear = LinearProbing()
quadratic = QuadraticProbing()

while True:

    print("\n========== HASH TABLE ==========")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Display All")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    # Separate Chaining
    if choice == 1:
        key = int(input("Enter key: "))
        chaining.insert(key)
        chaining.display()
```



```
        chaining.display()

    # Linear Probing
    elif choice == 2:
        key = int(input("Enter key: "))
        linear.insert(key)
        linear.display()

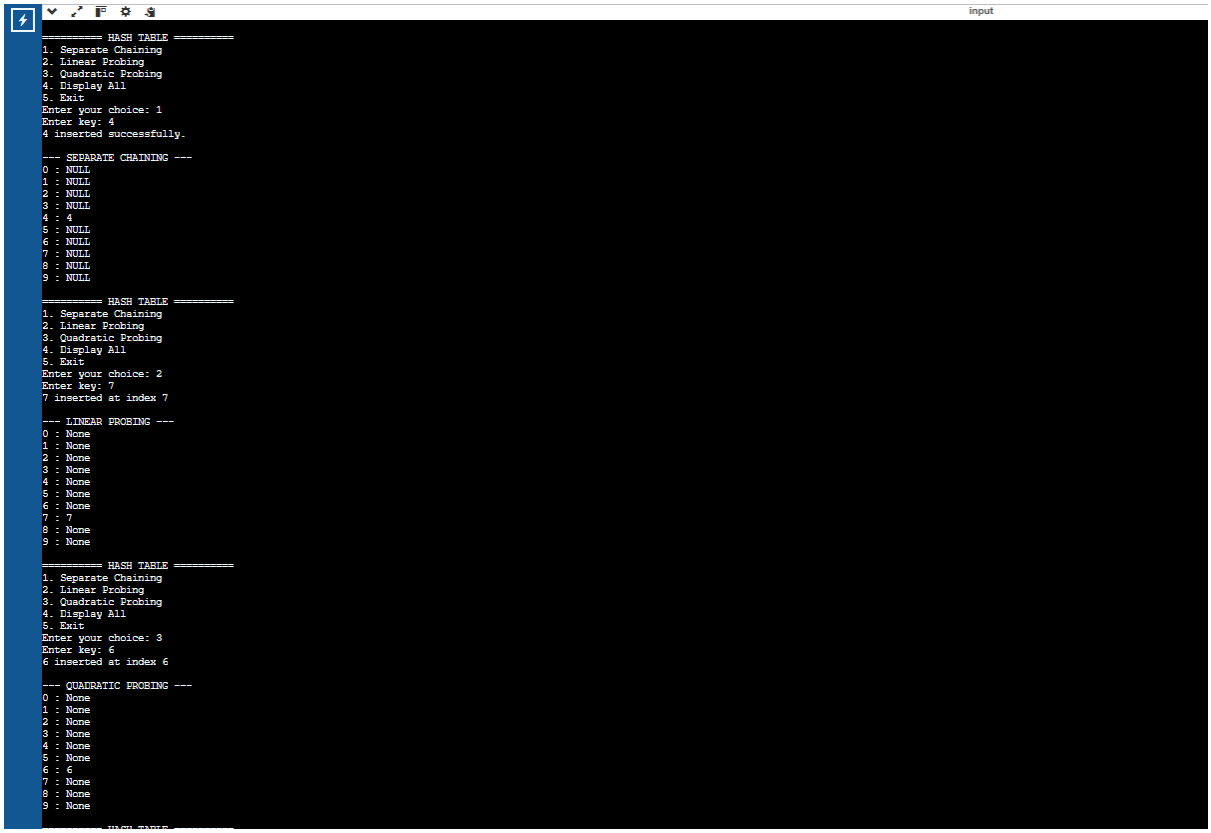
    # Quadratic Probing
    elif choice == 3:
        key = int(input("Enter key: "))
        quadratic.insert(key)
        quadratic.display()

    # Display all
    elif choice == 4:
        chaining.display()
        linear.display()
        quadratic.display()

    # Exit
    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice!")
```

OUTPUT:


input
========= HASH TABLE =========
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 1
Enter key: 4
4 inserted successfully.
--- SEPARATE CHAINING ---
0 : NULL
1 : NULL
2 : NULL
3 : NULL
4 : 4
5 : NULL
6 : NULL
7 : NULL
8 : NULL
9 : NULL
========= HASH TABLE =========
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 2
Enter key: 7
7 inserted at index 7
--- LINEAR PROBING ---
0 : None
1 : None
2 : None
3 : None
4 : None
5 : None
6 : None
7 : 7
8 : None
9 : None
========= HASH TABLE =========
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Display All
5. Exit
Enter your choice: 3
Enter key: 6
6 inserted at index 6
--- QUADRATIC PROBING ---
0 : None
1 : None
2 : None
3 : None
4 : None
5 : None
6 : 6
7 : None
8 : None
9 : None